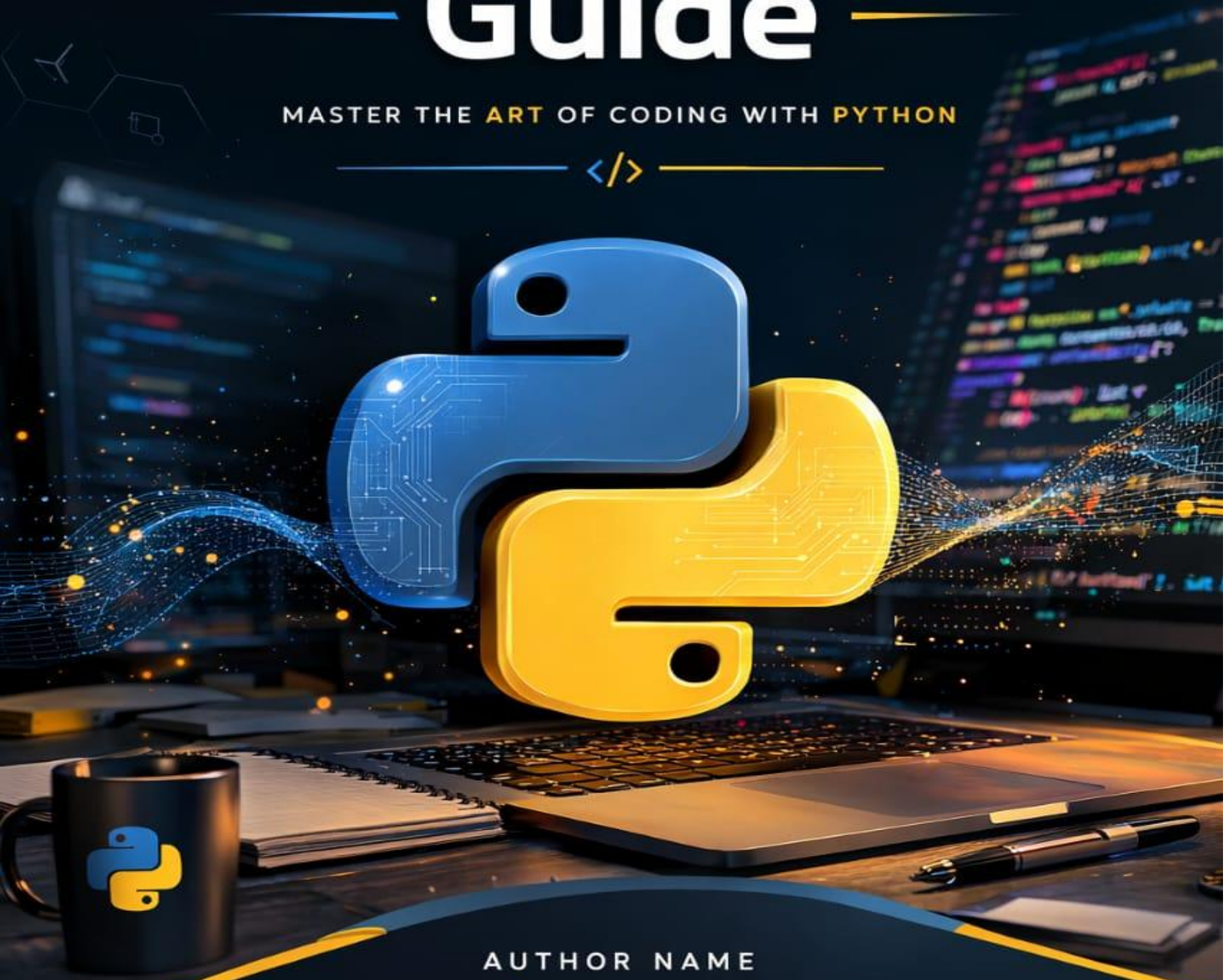
Adar's Python Guide

MASTER THE ART OF CODING WITH PYTHON

</>



AUTHOR NAME

Adar Dey

Author Introduction

My name is **Adar Dey**, and I come from **Patiya, Chattogram, Bangladesh**. From an early age, I developed a deep curiosity for **logic and mathematics**. Rather than simply memorizing formulas, I always enjoyed thinking differently and trying to solve problems in my **own unique ways**. Creating personal methods to understand and solve problems in **mathematics and physics** gradually shaped the way I approached learning.

I completed my college education at **Gachbaria Government College, Chandanaish, Chattogram** in **2021**. Later, in **2022**, I began my university journey at **Comilla University**, one of the public universities of Bangladesh. After adapting to university life, I realized that I needed to dedicate myself to something meaningful that would strengthen both my **knowledge and future career**. That realization led me toward the world of **programming**.

In the early days of my programming journey—before artificial intelligence became as dominant as it is today—I spent much of my time exploring **Data Structures and Algorithms (DSA)**, solving logical problems, and studying **competitive programming techniques**. I particularly enjoyed learning new algorithms and understanding how complex problems could be solved through elegant logical solutions.

Currently, I am a **third-year student** at my university, and during this time I have witnessed how **Artificial Intelligence is rapidly transforming every sector of technology and society**. As someone who believes strongly in adapting to change and continuously improving, I have developed a strong interest in **Artificial Intelligence and Machine Learning**.

To move forward in this field, one of the most important foundations is a strong command of **Python**, the language that powers a large portion of modern AI and machine learning technologies. While I have always admired **C++** for its efficiency, performance, and clarity—indeed many powerful frameworks and systems are built with it—I realized that mastering **Python** is essential for working effectively in the AI-driven world.

Because of this realization, I decided to dedicate myself to learning Python deeply and systematically. During this journey, I felt the need to organize my understanding in a clear and structured way—not only for myself but also for others who are beginning their programming journey.

This book "*Adar's Python Guide*" is the result of that journey.

It reflects my personal learning process, my curiosity about logical problem solving, and my commitment to adapting to the evolving world of technology. My goal through this book is to make Python learning **clear, logical, and approachable**, especially for students who enjoy understanding concepts deeply rather than simply memorizing syntax.

I believe that programming is not just about writing code—it is about **developing a way of thinking**, solving problems creatively, and building tools that can shape the future.

I sincerely hope that this book helps readers build a strong foundation in Python and inspires them to explore the fascinating world of programming, algorithms, and intelligent systems.

— Adar Dey

PYTHON

Adarsh Python Guide

Chapter: 1

Introduction to Python, Modules, and Comments

Adar's Python Guide

Question: What is Python Programming Language?

Answer: **Python** is a **programming language** used to tell a computer what to do.

→ You write instructions
→ Python reads them
→ Computer follows them

Key idea:
Python looks almost like **normal English**, so beginners can read and write it easily.

Example: print("Hello World")

This means:
→ "Show the text *Hello World* on the screen."

Question: Why Python Programming Language?

Answer: Python is popular because:

→ **Easy to learn** (simple syntax)
→ **Less code**, more work
→ Used in **Web**, **AI**, **Data Science**, **Automation**, **Backend**, **Games**
→ Great for **beginners** and also **professionals**

Simple comparison:

- Other languages: complex rules
- Python: focus on logic, not syntax

That's why Python is often the **first language** people learn.

Adar's Python Guide

Question: What is Python Interpreter?

Answer: The **Python interpreter** is a **program** that:

➞ Reads your Python code
➞ Translates it into machine instructions
➞ Executes it **line by line**

Important point:

Computers do NOT understand Python directly.
They only understand **machine code (0 and 1)**.

Question: How Python code runs on Hardware?

Answer:

Python Code (.py) → Python Interpreter → Machine Code (0 & 1) → CPU (Hardware) → Output (Screen / File)

Question: How to download Python and run it?

Answer:

Step 1: Download Python

Open Browser → Go to python.org → Download Python → Install Python → Check "Add to PATH" → Python Ready

Step 2: Install VS Code (Code Editor)

Open Browser → Go to code.visualstudio.com → Download VS Code → Install → Open VS Code

Step 3: Install Python Extension in VS Code

Open VS Code → Extensions Tab → Search "Python" → Install Extension → Ready to Code

Step 4: Run Python Code

Create file (example.py) → Write Python Code → Click Run / Terminal → Interpreter Executes → Output Shown

Adar's Python Guide

Question: What is a module in Python?

Answer:

A **module** is a **file** that contains Python code.

➞ Functions
➞ Variables
➞ Classes

Purpose: Reuse code instead of writing everything again.

Example:

```python
import math
print(math.sqrt(16))
```

- math is a **module**
- sqrt is a function inside it

Download a module in Mac Terminal/Linux Bash/Windows CMD : pip install **module_Name**

Question: What is REPL in Python?

Answer:

REPL means:

➞ **R**ead
➞ **E**valuate
➞ **P**rint
➞ **L**oop

It is an **interactive Python mode**.

Type Python Code → Read → Evaluate → Print Result → Loop Again

Adar's Python Guide

Question: What are the comments in Python?

Answer:

Comments are **notes** for humans.
Python **ignores** comments.

🔹 Single-Line Comment: Use #

```python
# This is a comment
x = 5  # storing value in x
```

🔹 Multi-Line Comment: Use triple quotes (''' or """)

```python
"""
This is a multi-line comment.
Python ignores this.
Used for explanation.
"""
```

Problem: Install the Python package **Pyjokes** using `pip` and write a Python program to fetch and print jokes using the package. Run it in your Python environment (e.g., Python terminal or VS Code).

Solution:

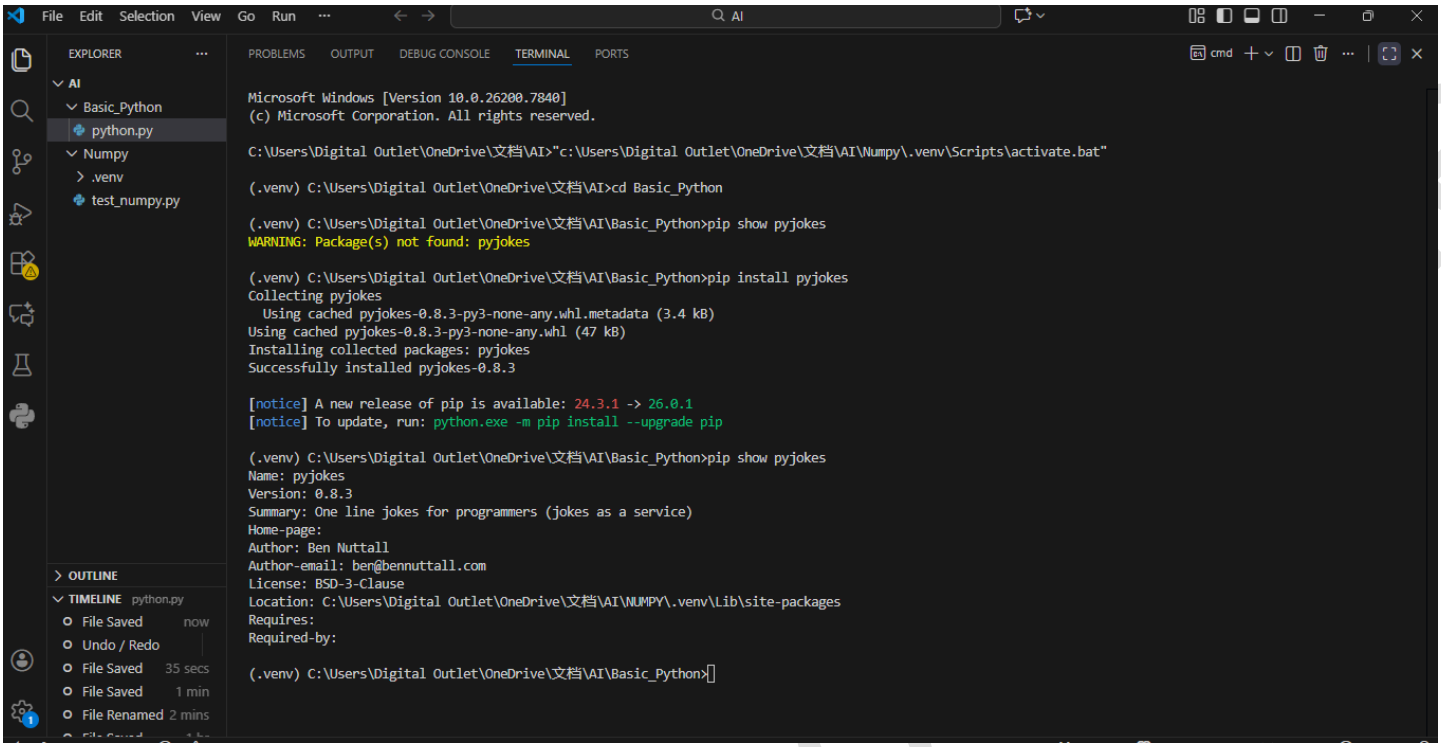
Step 1: Check if the `pyjokes` module exists by using `pip show pyjokes`.

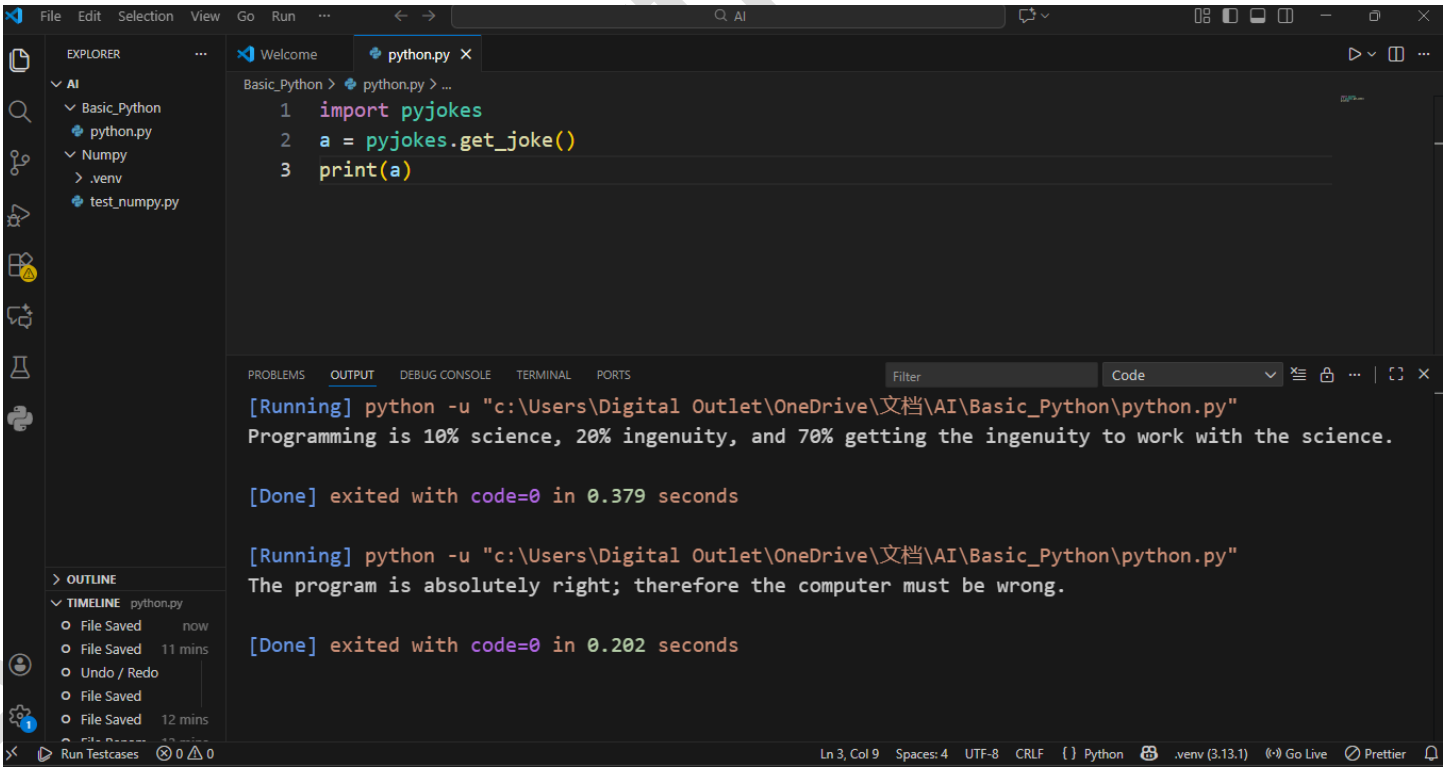
Step 2: If it doesn't exist, install it using `pip install pyjokes`.

Step 3: Check again to confirm that `pyjokes` is installed using `pip show pyjokes`.

Step 4: Use the `pyjokes` module to get a joke and print it.

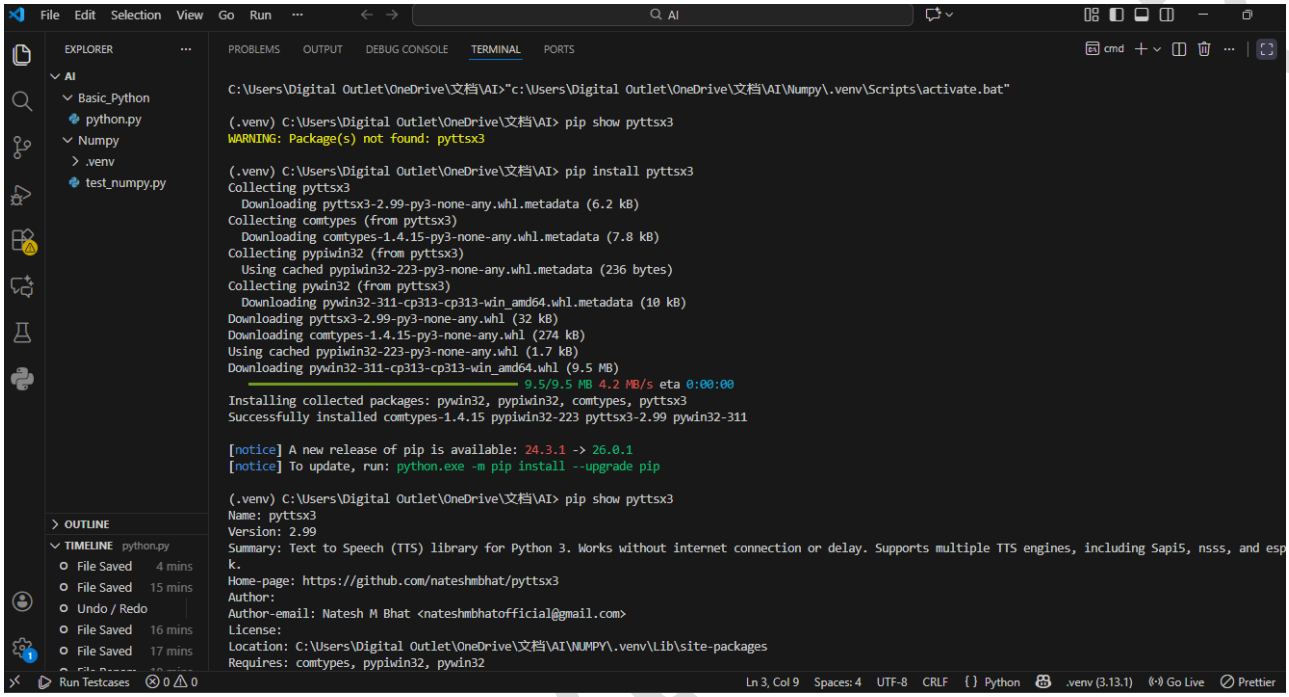
Adar's Python Guide

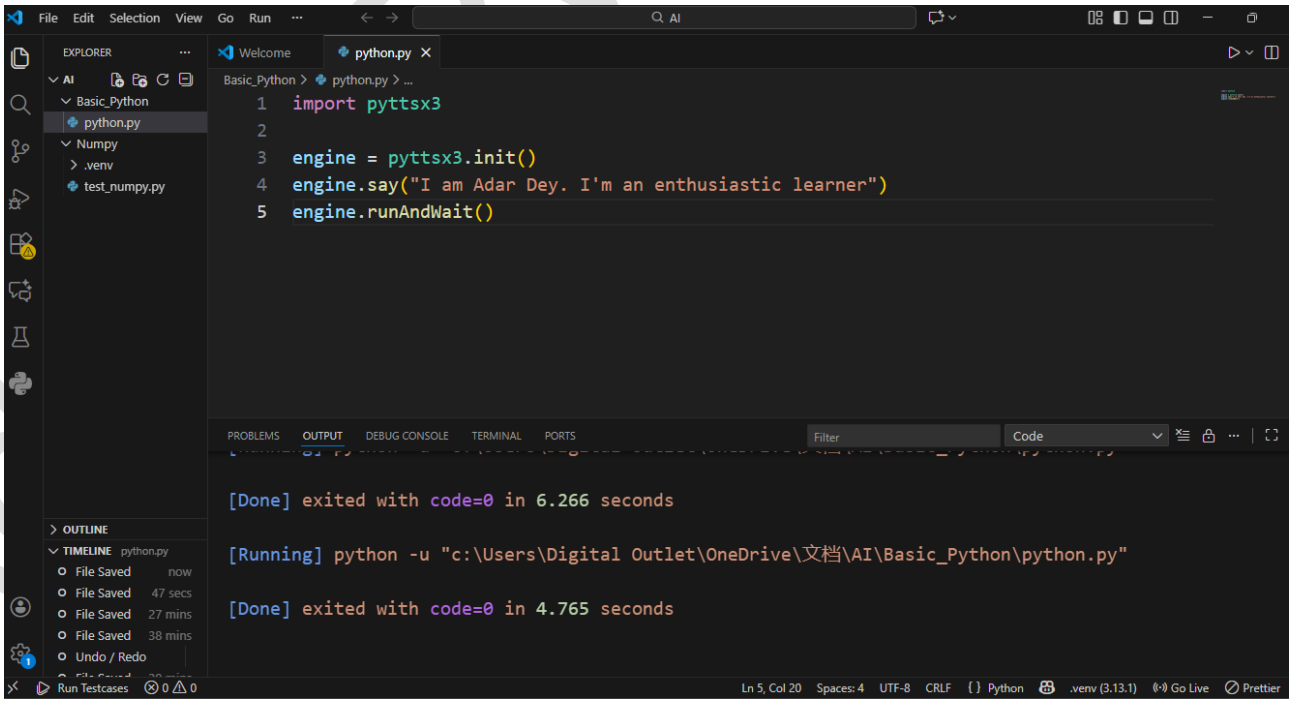

File Edit Selection View Go Run
AI
EXPLORER
AI
Basic_Python
python.py
Numpy
.venv
test_numpy.py
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
cmd
Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.
C:\Users\Digital Outlet\OneDrive\文档\AI>"c:\Users\Digital Outlet\OneDrive\文档\AI\Numpy\.venv\Scripts\activate.bat"
(.venv) C:\Users\Digital Outlet\OneDrive\文档\AI>cd Basic_Python
(.venv) C:\Users\Digital Outlet\OneDrive\文档\AI\Basic_Python>pip show pyjokes
WARNING: Package(s) not found: pyjokes
(.venv) C:\Users\Digital Outlet\OneDrive\文档\AI\Basic_Python>pip install pyjokes
Collecting pyjokes
Using cached pyjokes-0.8.3-py3-none-any.whl.metadata (3.4 kB)
Using cached pyjokes-0.8.3-py3-none-any.whl (47 kB)
Installing collected packages: pyjokes
Successfully installed pyjokes-0.8.3
[notice] A new release of pip is available: 24.3.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) C:\Users\Digital Outlet\OneDrive\文档\AI\Basic_Python>pip show pyjokes
Name: pyjokes
Version: 0.8.3
Summary: One line jokes for programmers (jokes as a service)
Home-page:
Author: Ben Nuttall
Author-email: ben@bennuttall.com
License: BSD-3-Clause
Location: C:\Users\Digital Outlet\OneDrive\文档\AI\NUMPY\.venv\Lib\site-packages
Requires:
Required-by:
(.venv) C:\Users\Digital Outlet\OneDrive\文档\AI\Basic_Python>
OUTLINE
TIMELINE python.py
File Saved now
Undo / Redo
File Saved 35 secs
File Saved 1 min
File Renamed 2 mins


File Edit Selection View Go Run
AI
EXPLORER
AI
Basic_Python
python.py
Numpy
.venv
test_numpy.py
Welcome
python.py
Basic_Python > python.py > ...
1 import pyjokes
2 a = pyjokes.get_joke()
3 print(a)
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Filter
Code
[Running] python -u "c:\Users\Digital Outlet\OneDrive\文档\AI\Basic_Python\python.py"
Programming is 10% science, 20% ingenuity, and 70% getting the ingenuity to work with the science.
[Done] exited with code=0 in 0.379 seconds
[Running] python -u "c:\Users\Digital Outlet\OneDrive\文档\AI\Basic_Python\python.py"
The program is absolutely right; therefore the computer must be wrong.
[Done] exited with code=0 in 0.202 seconds
OUTLINE
TIMELINE python.py
File Saved now
File Saved 11 mins
Undo / Redo
File Saved
File Saved 12 mins
Run Testcases
Ln 3, Col 9 Spaces: 4 UTF-8 CRLF Python .venv (3.13.1) Go Live Prettier

Problem: Install the Python package **pyttsx3** using `pip`, and write a Python program to **convert text to speech** so that you can **listen to audio on your machine**. Run the program in your Python environment (e.g., **Python terminal** or **VS Code**).

Solution:





Adar Dey

Chapter 1 – Practice Set

1. Write a Python program to **print the poem "Twinkle, Twinkle, Little Star"**.
2. Use **REPL** to **print the multiplication table of 5**.
3. Install an **external Python module** and use it to **perform any operation of your choice**.
4. Write a Python program to **print the contents of a directory** using the `os` module. *(Search online for the function that does this.)*
5. **Add comments** to the program written in Problem 4 to explain what each part of the code does.

Adar's Python Guide

Chapter: 2

Variables, Data Types, Operators and Input

Adar's Python Guide

Question: What is a variable? What are the variable conventions in python?

Answer:

A **variable** is a **name** that refers to a value stored in the computer's memory. Think of a variable as a **label** attached to some data.

Example:

```python
x = 10
```

Here:

- x → variable name
- = → assignment operator
- 10 → value stored in memory

Meaning: "Store the value 10 inside a variable named x."

 The name of a variable will always be a word.
That means a variable name cannot contain spaces. We cannot write a variable name using more than one separate word.
If we want to use two or more words, we must remove the spaces and write them together as a single word.
Another way is to use an underscore (_) between two words while omitting the space.
This helps to make the variable name readable and valid in Python.

 The first letter of a variable name must follow specific rules.
The first character of a variable name must be an uppercase letter (A–Z), a lowercase letter (a–z), or an underscore (*_*).
The first letter cannot be a number (0–9) or any other symbol.
However, after the first letter, numbers (0–9), uppercase letters, lowercase letters, and underscores (_) can be used as the remaining characters of the variable name.

 No special symbols are allowed in variable names.
Variable names cannot contain any special characters such as dollar sign, caret, ampersand, at-sign, or similar symbols.
Only letters, numbers, and underscores are allowed according to Python naming rules.

Adar's Python Guide

 Python is a case-sensitive programming language.
This means Python treats uppercase and lowercase letters differently.
If the same variable name is written once using uppercase letters and once using lowercase letters, Python will consider them as two different variables.
Therefore, programmers must be careful while using uppercase and lowercase letters in variable names.

 Keywords cannot be used as variable names.
Keywords are reserved or saved words in the Python programming language.
These words have special meanings and are already defined by Python.
We cannot use keywords as the names of variables, functions, or any other identifiers.
In Python version 3.10, there are a total of 35 reserved keywords.
Examples include words that represent logical values and control statements.

Incorrect Naming of Variable	Correct Naming of Variable
My Variable	MyVariable or My_Variable
National ID	NationalID or National_ID
this variable is cool	thisvariableiscool or this_variable_is_cool
z!yan	zyan or z_yan
9abc	abc9
$variable	_variable
@My_name	My_name
print	Keywords cannot be used as variable names. However, Myprint or print1 is correct.

Exercise Table

Adar's Python Guide

Variable Name (Given)	Your Answer (Correct / Incorrect+Correction)
student name	
StudentName	
student_Name	
1student	
student1	
total-marks	
total_marks	
TotalMarks	
total$marks	
_totalMarks	
@username	
user_name_1	
user name 1	
9marks	
marks9	
average%score	
average_score	
myVariable	
Myvariable	
myvariable	
my variable is good	
my_variable_is_good	
print	
Print	
print1	
class	
className	
None	
noneValue	
False	
false_value	
total__marks	
init	
first-name	
first_name	
score@home	
score_home	
student#id	
student_id	
final.result	
final_result	
for	
forLoop	
while	
while_count	
123value	
value123	
__	
__value	

Solution of Exercise:

student name – Incorrect (student_name),
StudentName – Correct,
student_Name – Correct,
1student – Incorrect (student1),
student1 – Correct,
total-marks – Incorrect (total_marks),
total_marks – Correct,
TotalMarks – Correct,
total$marks – Incorrect (total_marks),
_totalMarks – Correct,
@username – Incorrect (username),
user_name_1 – Correct,
user name 1 – Incorrect (user_name_1),
9marks – Incorrect (marks9),
marks9 – Correct,
average%score – Incorrect (average_score),
average_score – Correct,
myVariable – Correct,
Myvariable – Correct,
myvariable – Correct,
my variable is good – Incorrect (my_variable_is_good),
my_variable_is_good – Correct,
print – Incorrect (print1),
Print – Correct,
print1 – Correct,
class – Incorrect (class_name),
className – Correct,
None – Incorrect (none_value),
noneValue – Correct,
False – Incorrect (false_value),
false_value – Correct,
total__marks – Correct,
init – Correct,
first-name – Incorrect (first_name),
first_name – Correct,
score@home – Incorrect (score_home),
score_home – Correct,
student#id – Incorrect (student_id),
student_id – Correct,
final.result – Incorrect (final_result),
final_result – Correct,
for – Incorrect (for_loop),
forLoop – Correct,
while – Incorrect (while_count),
while_count – Correct,
123value – Incorrect (value123),
value123 – Correct,
__ – Correct,
__value – Correct

Arar's Python Guide

Question: How to declare variable in Python?

Answer:

Declaring different types of variable:

```python
age = 21          # integer
price = 99.5      # float
name = "Adar"     # string
is_student = True  # boolean
```

Declaring multiple variables:

```python
a = b = c = 5
```

Different values to multiple variables:

```python
x, y, z = 1, 2, 3
```

Reassigning a variable:

```python
x = 10
x = 20
```

Adar's Python Guide

Question: Write down details about Data Types in python?

Answer:

A **data type** tells Python **what kind of data** a variable is storing.

Python has **several built-in data types**, mainly divided into **numeric, sequence, mapping, set, and boolean types**.

Table of Common Python Data Types

Data Type	Example	Description	Notes
int	`x = 10`	Stores **whole numbers**	Positive or negative
float	`y = 3.14`	Stores **decimal numbers**	Example: 0.5, -2.7
complex	`z = 2 + 3j`	Stores **complex numbers**	`j` is imaginary unit
str	`name = "Adar"`	Stores **text / string**	Can use single or double quotes
bool	`flag = True`	Stores **True / False**	Used in conditions
list	`lst = [1, 2, 3]`	Stores **ordered collection**	Can contain mixed types, mutable
tuple	`tup = (1, 2, 3)`	Stores **ordered collection**	Immutable (cannot change)
set	`s = {1, 2, 3}`	Stores **unique unordered items**	No duplicates, mutable
frozenset	`fs = frozenset([1,2,3])`	Immutable version of set	Cannot add/remove elements
dict	`d = {"name":"Adar", "age":21}`	Stores **key-value pairs**	Keys are unique, unordered in Python <3.7
NoneType	`x = None`	Represents **no value / null**	Used as placeholder

Dynamic typing: Python automatically detects the data type when you assign a value.

```python
x = 5       # int
x = 3.5     # float (no error)
```

Mutable vs Immutable:

- Mutable → can change value (list, set, dict)
- Immutable → cannot change value (int, float, str, tuple, frozenset)

Type checking: Use type() function to see a variable's type.

```python
x = 10
print(type(x))  # <class 'int'>
```

Adar's Python Guide

Type	Mutable	Example
int	✗	10
float	✗	3.14
complex	✗	2+3j
str	✗	"Hello"
list	✓	[1,2,3]
tuple	✗	(1,2,3)
set	✓	{1,2,3}
frozenset	✗	frozenset([1,2,3])
dict	✓	{"a":1}
bool	✗	True / False
NoneType	✗	None

Question: What is Typecasting? And why is it important in Python?

Answer:

"Typecasting = changing the type of data temporarily to match the operation"

```python
x = "10"
y = "20"

# Strings + strings → concatenation
print(x + y)   # Output: 1020

# Convert to integers for addition
x = int(x)
y = int(y)
print(x + y)   # Output: 30

# Float conversion
z = float(x)
print(z)       # Output: 10.0

# Boolean conversion
print(bool(0)) # Output: False
print(bool(5)) # Output: True
```

Python Guide

Question: Write down details about Operators in Python?

Answer:

An **operator** is a **symbol** that tells Python to **perform a specific operation** on one or more values.

Operator Type	Operator	Meaning	Example	Output
Arithmetic	+	Addition	5 + 3	8
	-	Subtraction	5 - 3	2
	*	Multiplication	5 * 3	15
	/	Division	5 / 2	2.5
	//	Floor Division	5 // 2	2
	%	Modulus (remainder)	5 % 2	1
	**	Exponentiation	2 ** 3	8
Comparison	==	Equal to	5 == 3	False
	!=	Not equal to	5 != 3	True
	>	Greater than	5 > 3	True
	<	Less than	5 < 3	False
	>=	Greater or equal	5 >= 5	True
	<=	Less or equal	3 <= 5	True
Assignment	=	Assign value	x = 5	5
	+=	x = x + y	x = 5; x += 3	8
	-=	x = x - y	x = 5; x -= 2	3
	*=	x = x * y	x = 5; x *= 2	10
	/=	x = x / y	x = 6; x /= 2	3.0
	//=	x = x // y	x = 5; x //= 2	2
	%=	x = x % y	x = 5; x %= 3	2
	**=	x = x ** y	x = 2; x **= 3	8
Logical	and	True if both True	(5 > 2) and (3 < 5)	True
	or	True if at least one True	(5 < 2) or (3 < 5)	True
	not	Reverse True/False	not(5 > 2)	False
Identity	is	True if same object	x = [1]; y = x; x is y	True
	is not	True if not same object	x = [1]; y = [1]; x is not y	True
Membership	in	True if exists in sequence	'a' in 'apple'	True
	not in	True if does not exist	'b' not in 'apple'	True
Bitwise	&	AND	5 & 3	1
	`	`	OR	`5
	^	XOR	5 ^ 3	6
	~	NOT	~5	-6
	<<	Left shift	5 << 1	10
	>>	Right shift	5 >> 1	2

Adarsh Python Guide

Question: How to take input from users in Python?

Answer:

Python uses the **input()** function to take input from the user.

Basic syntax : `variable = input("message")`

- input() **always takes input as a string**
- The text inside quotes is a **prompt message**
- To use numbers, you must **typecast** the input

Taking String Input :

```
name = input("Enter your name: ")
print(name)
```

Taking Integer Input:

```
age = int(input("Enter your age: "))
print(age)
```

Common Beginner Mistake ❌

Python

```
x = input("Enter number: ")
print(x + 5)   # ERROR
```

✔ Correct way:

Python

```
x = int(input("Enter number: "))
print(x + 5)
```

Python Guide

Chapter 2 – Practice Set

1. Write a Python program to add two numbers.

2. Write a Python program to find the remainder when one number is divided by another number.

3. Check the type of a variable assigned using the `input()` function.

4. Use the comparison operator to find out whether a given variable `a` is greater than `b` or not. Take `a = 34 and b = 80.`

5. Write a Python program to find the average of two numbers entered by the user.

6. Write a Python program to calculate the square of a number entered by the user.

Adar's Python Guide

Chapter: 3

Strings

Adar's Python Guide

Question: How to take input from users in Python?

Answer:

A **string** is a **sequence of characters**.

Characters can be:

- Letters → a b c
- Numbers → 1 2 3
- Symbols → @ # $
- Spaces → " "

In Python, a string is written inside **quotes**.

Example: `name = "Adar"`

Here:

- "Adar" is a **string**
- A, d, a, r are **characters**

Index from Beginning of a String (Positive Index)

word = "Python"

```
Character :  P   y   t   h   o   n
Index     :  0   1   2   3   4   5
```

Index from Ending of a String (Negative Index)

word = "Python"

```
Character :  P   y   t   h   o   n
Index     : -6  -5  -4  -3  -2  -1
```

🔑 "Strings are immutable but reassignable"

Adar's Python Guide

```python
# ======================================================
# PYTHON STRING – COMPLETE BEGINNER CODE NOTE
# ======================================================

# 1. Creating Strings
s1 = "Hello"
s2 = 'Python'
s3 = """Learning Python"""

# ------------------------------------------------------
# 2. String is IMMUTABLE but REASSIGNABLE
# ------------------------------------------------------

# ❌ Not allowed (immutable)
# s1[0] = 'h'   # Error

# ✅ Allowed (reassignable)
s1 = "hello"

# ------------------------------------------------------
# 3. Length of a String
# ------------------------------------------------------
text = "Python"
print(len(text))   # 6

# ------------------------------------------------------
# 4. Indexing
# ------------------------------------------------------
# Positive indexing (from beginning)
print(text[0])   # P
print(text[3])   # h

# Negative indexing (from end)
print(text[-1])  # n
print(text[-3])  # h
```

```python
# ------------------------------------------------------
# 5. String Slicing
# ------------------------------------------------------
print(text[0:4])    # Pyth
print(text[2:])     # thon
print(text[:3])     # Pyt
print(text[-3:])    # hon

# Format:
# string[start : end]  -> end index is NOT included

# ------------------------------------------------------
# 6. Looping Through a String
# ------------------------------------------------------
for ch in "Python":
    print(ch)

# ------------------------------------------------------
# 7. String Concatenation
# ------------------------------------------------------
a = "Hello"
b = "World"
print(a + " " + b)   # Hello World

# ------------------------------------------------------
# 8. String Repetition
# ------------------------------------------------------
print("Hi " * 3)     # Hi Hi Hi

# ------------------------------------------------------
# 9. Common String Methods
# ------------------------------------------------------
msg = "  Python Programming  "

print(msg.upper())     # PYTHON PROGRAMMING
print(msg.lower())     # python programming
print(msg.strip())     # Python Programming
print(msg.replace("Python", "Java"))

# Split string into list
line = "Python is easy"
print(line.split())    # ['Python', 'is', 'easy']
```

Python Guide

```python
# ------------------------------------------------------
# 10. Checking Substring
# ------------------------------------------------------
text = "Python programming"
print("Python" in text)   # True
print("Java" in text)     # False

# ------------------------------------------------------
# 11. String Formatting (Best Way)
# ------------------------------------------------------
name = "Adar"
age = 21
print(f"My name is {name} and I am {age} years old")

# ------------------------------------------------------
# 12. Escape Characters
# ------------------------------------------------------
print("Hello\nWorld")        # New line
print("He said \"Hello\"")  # Quotes inside string

# ------------------------------------------------------
# 13. Common Beginner Mistake
# ------------------------------------------------------
age = 21

# ❌ Error
# print("Age is " + age)

# ✅ Correct
print("Age is " + str(age))

# ======================================================
# END OF STRING NOTES
# ======================================================
```

Your Task: Learn string methods by reading the documentation and exploring strings thoroughly.

Question: What are the Escape Sequence Characters?

Answer:

An **escape sequence character** is a **special combination of characters** used inside a string to represent something that **cannot be typed normally** (like a new line or a tab).

Escape Sequence	Meaning	Simple Explanation
`\n`	New line	Moves text to next line
`\t`	Tab space	Adds horizontal space
`\\`	Backslash	Prints \
`\'`	Single quote	Prints '
`\"`	Double quote	Prints "
`\r`	Carriage return	Moves cursor to start of line
`\b`	Backspace	Removes one character
`\f`	Form feed	Page break (rarely used)

Python

```python
print("Hello\nWorld")
```

Output:

Code

```
Hello
World
```

Adar's Python Guide

Chapter 3 – Practice Set

1. Write a Python program to display a user-entered name followed by **"Good Afternoon"** using the input function.

2. Write a Python program to fill in the following letter template using a **name** and **date**:

```
Dear <Name>,
You are selected!
<Date>
```

3. Write a Python program to detect **double spaces** in a string.

4. Write a Python program to replace **double spaces** from problem 3 with **single spaces**.

5. Write a Python program to format the following letter using **escape sequence characters**:

```
Dear Harry, this python course is nice. Thanks!
```

Adar's Python Guide

Chapter: 4
Lists and Tuples

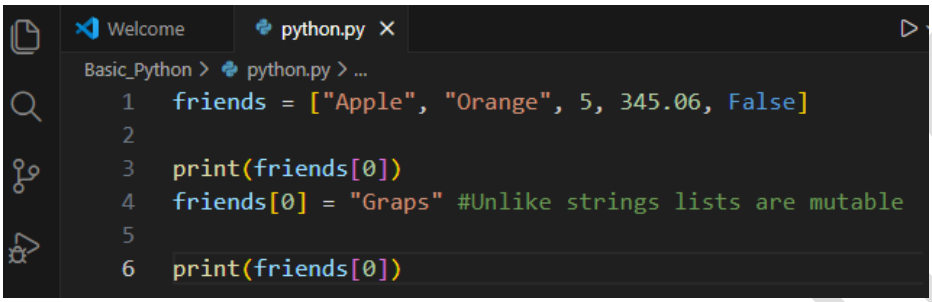
Adar's Python Guide

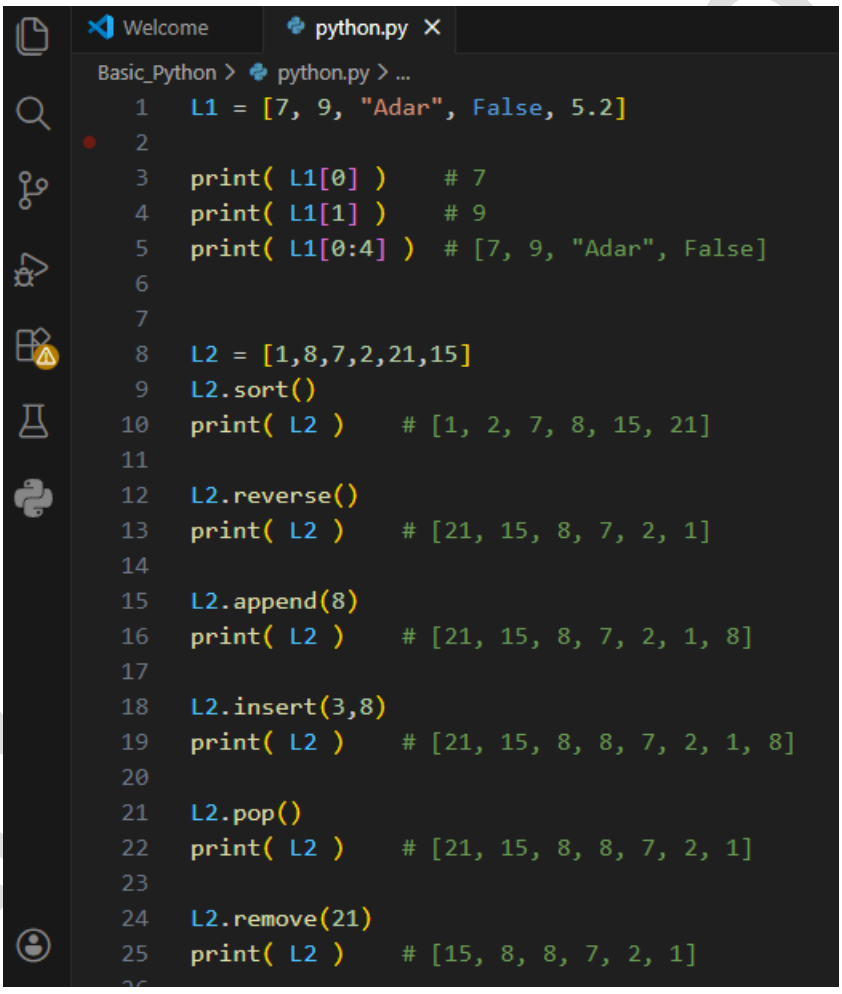
Question: What are Lists in Python?

Answer:

A **list** in Python is a **collection of items** stored in a **single variable**.

- It can store **multiple values** together
- Items can be of **any data type**
- Lists are **ordered**, **changeable (mutable)**, and **allow duplicates**





Your Task: Learn lists methods by reading the documentation and exploring lists thoroughly.

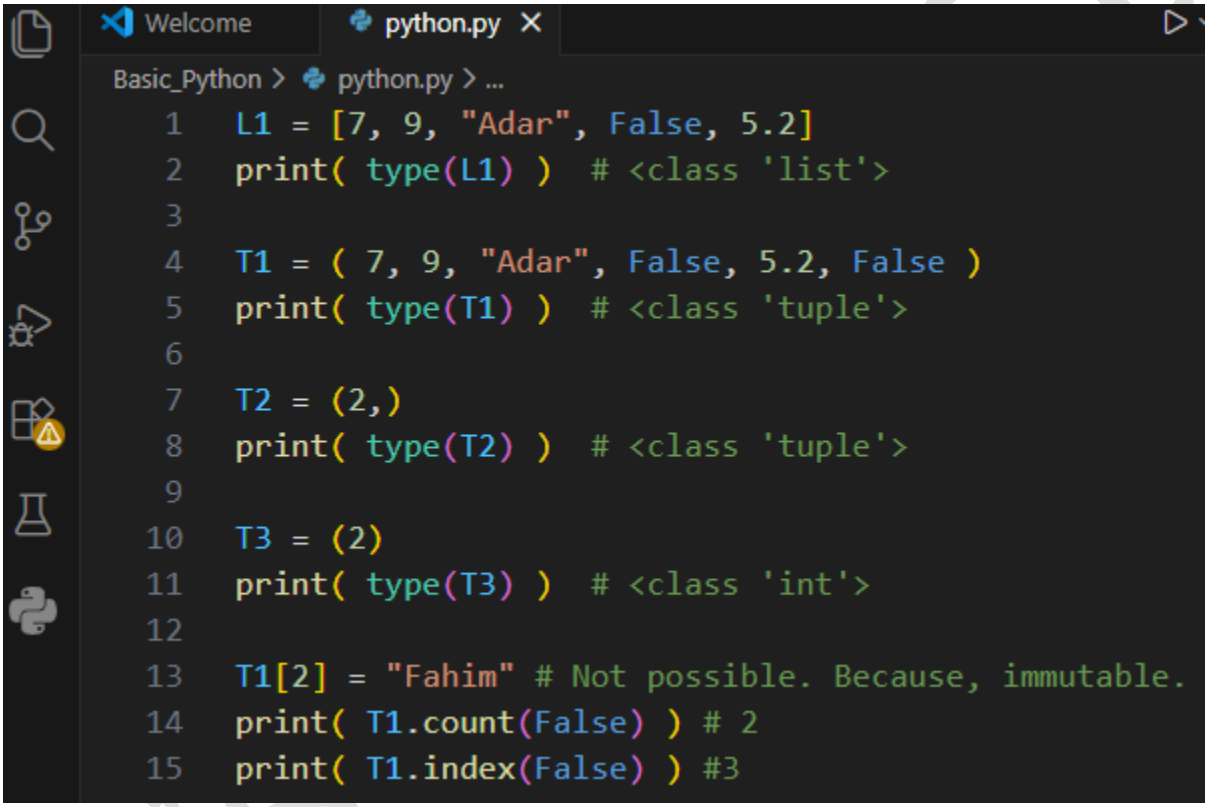
Adar Guide

Question: What are Tuples in Python?

Answer:

A **tuple** in Python is a **collection of items** that is:

- **Ordered** → items have fixed positions
- **Immutable** → items cannot be changed after creation
- **Allows different data types**
- **Written using parentheses ()**



Your Task: Learn tuples methods by reading the documentation and exploring tuples thoroughly.

Adar's Guide

Chapter 4 – Practice Set

1. Write a program to store **seven fruits in a list** entered by the user.

2. Write a program to **accept marks of 6 students** and display them in a **sorted manner**.

3. Check that a **tuple type cannot be changed in Python**.

4. Write a program to **sum a list with 4 numbers**.

5. Write a program to **count the number of zeros in the following tuple**:

```
a = (7, 0, 8, 0, 0, 9)
```

Adar's Python Guide

Chapter: 5
Dictionary and Sets

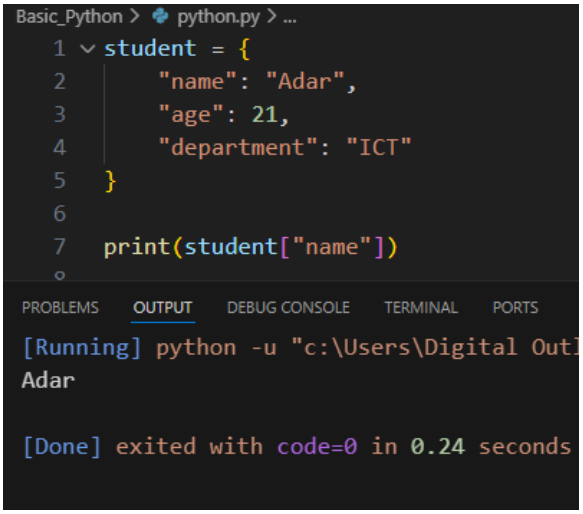
Adar's Python Guide

Question: What is a Dictionary in Python?

Answer:

A **dictionary** in Python is a **collection of data stored as key–value pairs**.

It is used when you want to **associate one value with another value**.



Key	Value
name	Adar
age	21
department	ICT

Important Properties of Dictionary

1. Stored as key–value pairs
2. Unordered (in concept)
3. Mutable and indexed
4. Keys must be unique
5. Values can be anything

Adar's Python Guide

Welcome | python.py

Basic_Python > python.py > ...

```python
student = {
    "name": "Adar",
    "age": 21,
    "department": "ICT"
}
print(student.items())
print(student.keys())
print(student.values())

student.update({"age":22, "varsity":"Comilla"})
print(student)

print(student.get("department"))

print(student.get("attribute"))
print(student["attribute"])

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[Running] python -u "c:\Users\Digital Outlet\OneDrive\文档\AI\Basic_Python\python.py"
dict_items([('name', 'Adar'), ('age', 21), ('department', 'ICT')])
dict_keys(['name', 'age', 'department'])
dict_values(['Adar', 21, 'ICT'])
{'name': 'Adar', 'age': 22, 'department': 'ICT', 'varsity': 'Comilla'}
ICT
None
Traceback (most recent call last):
  File "c:\Users\Digital Outlet\OneDrive\\u6587\u6863\AI\Basic_Python\python.py", line 16, in <module>
    print(student["attribute"])
          ~~~~~~~^^^^^^^^^^^^^
KeyError: 'attribute'

[Done] exited with code=1 in 0.245 seconds
```

Your Task: Learn Dictionary methods by reading the documentation and exploring Dictionary thoroughly.

Adar's Code

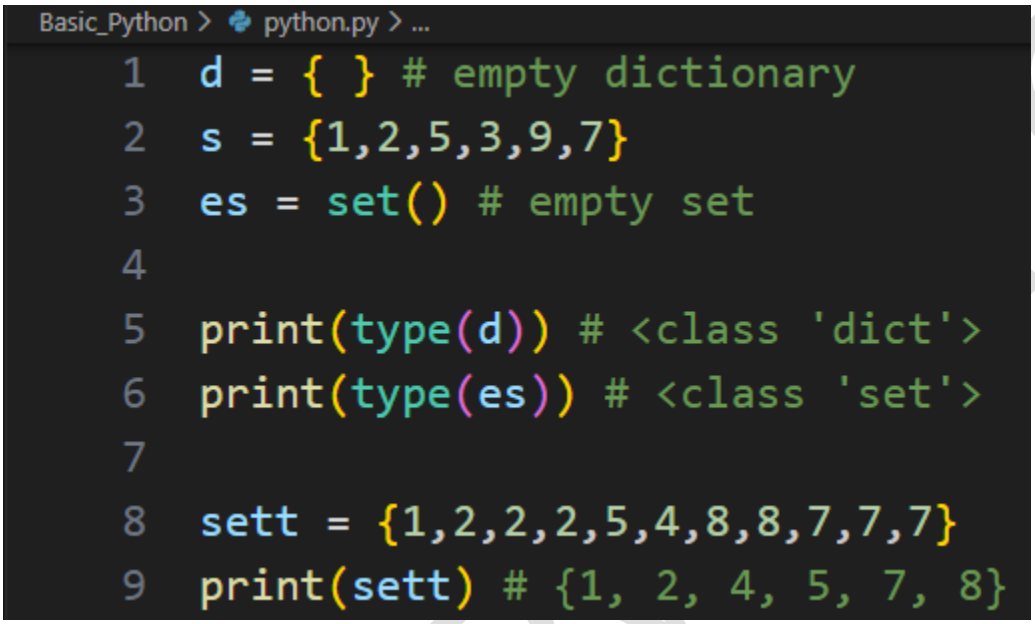
Question: What is a Set in Python?

Answer:

A **set** in Python is a **collection of unique elements**.

It is used when you want to **store items without duplicates**.

A set is written using **curly braces** {}.



Important Properties of Sets

1. Unordered and Unindexed
2. No duplicates values
3. Mutable (You **can add or remove elements** from a set.)
4. Elements must be immutable (Items inside a set must be **immutable**.)

Adar's Py Guide

```python
Basic_Python > python.py > ...
s = {1,5,6,4,7,"python"}
s.add(74)
s.remove(1)
# s.pop() # It removes random element form set

print(s)

p = {4,"hello"} # {4, 5, 6, 7, 'python', 74}

a = p.union(s)
print(a) # {'python', 4, 5, 6, 7, 74, 'hello'}

b = s.intersection(p)
print(b) # {4}

s.clear() # Empty set

print(a - b) # {'hello', 5, 6, 7, 74, 'python'}
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
[Running] python -u "c:\Users\Digital Outlet\OneDrive\文
{'python', 4, 5, 6, 7, 74}
{'python', 4, 5, 6, 7, 74, 'hello'}
{4}
{'python', 5, 6, 7, 74, 'hello'}

[Done] exited with code=0 in 0.136 seconds
```

Your Task: Learn Set methods by reading the documentation and exploring Set thoroughly.

Adar's Python Guide

Chapter 5 – Practice Set

1. Write a program to create a **dictionary of Bangla words** with values as their **English translations**. Provide the user with an option to **look it up**.

2. Write a program to **input eight numbers from the user** and display **all the unique numbers** (once).

3. Can we have a **set with 18 (int)** and **"18" (str)** as values in it?

4. What will be the **length of the following set S** after these operations?

```
S = set()
S.add(20)
S.add(20.0)
S.add("20")
```

5. S = {}
What is the **type of S**?

6. Create an **empty dictionary**. Allow **4 friends** to enter their **favorite language as value** and use their **names as keys**. Assume that the names are unique.

7. If the **names of two friends are the same**, what will happen to the program in **problem 6**?

8. If the **languages of two friends are the same**, what will happen to the program in **problem 6**?

9. Can you **change the values inside a list which is contained in a set S**?

```
S = {8, 7, 12, "Harry", [1, 2]}
```

Adar's Python Guide

Chapter: 6
Conditional Expressions

Adar's Python Guide

Question: What are the conditionals in Python?

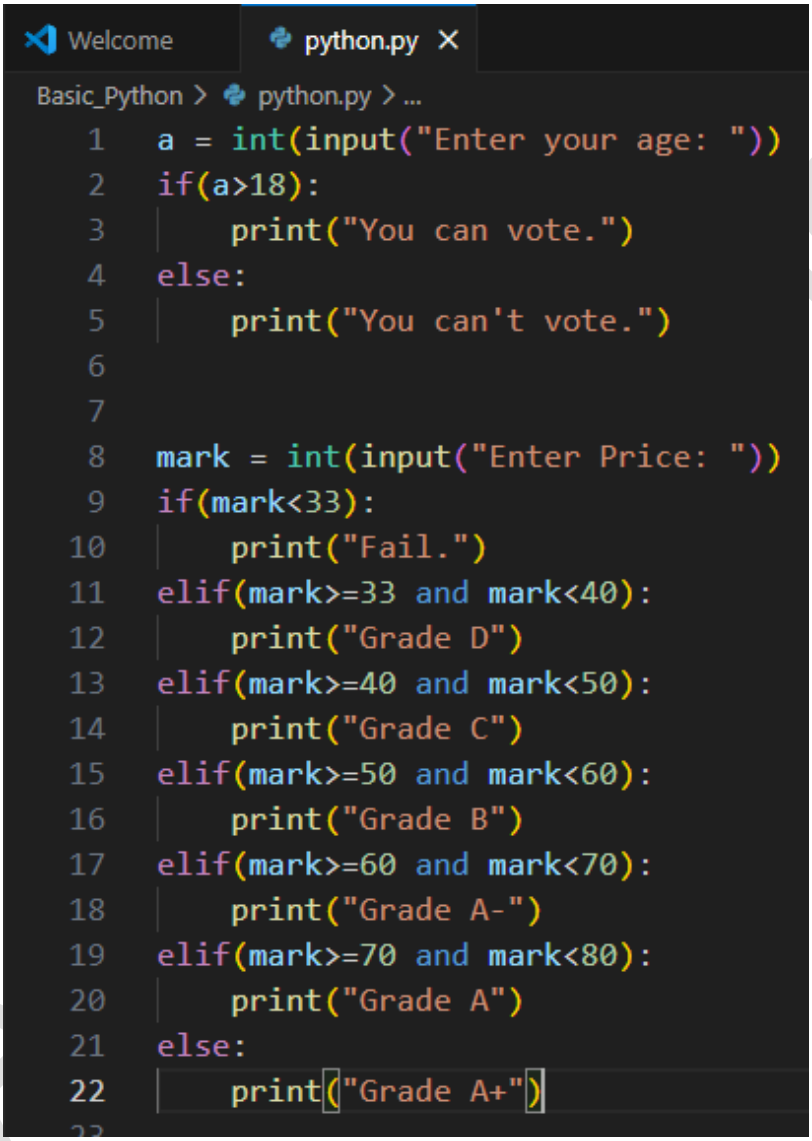
Answer:

Conditionals are used to make **decisions in a program**.

They allow the program to **execute different code depending on a condition**.

In simple words:

Conditionals tell the program **what to do if something is true or false**.



Your Task: Learn match-case (introduced in python 3.10) by reading the documentation and exploring it thoroughly.

Python Guide

Chapter 6 – Practice Set

1. Write a program to **find the greatest of four numbers** entered by the user.

2. Write a program to find out **whether a student has passed or failed** if it requires **a total of 40% and at least 33% in each subject** to pass. Assume **3 subjects** and take the **marks as input from the user**.

3. A **spam comment** is defined as a text containing the following keywords:
   "Make a lot of money", "buy now", "subscribe this", "click this".
   Write a program to **detect these spam comments**.

4. Write a program to find whether a **given username contains less than 10 characters or not**.

5. Write a program which finds out whether a **given name is present in a list or not**.

6. Write a program to **calculate the grade of a student** from his marks using the following scheme:

- **90 – 100 → Ex**
- **80 – 90 → A**
- **70 – 80 → B**
- **60 – 70 → C**
- **50 – 60 → D**
- **< 50 → F**

7. Write a program to find out whether a **given post is talking about "Adar" or not**.

Adar's Python Guide

Chapter: 7

Loops In Python

Adar's Python Guide

Question: What are Loops in Python and write about their types in detail?

Answer:

A **loop** is used to **repeat a block of code multiple times**.

Instead of writing the same code again and again, we use a **loop**.

A loop allows a program to **execute a set of instructions repeatedly**.

There are **two main loops** in Python:

1. **for loop**
2. **while loop**

1 `for` Loop

A **for loop** is used to iterate over a **sequence** such as:

- list
- tuple
- string
- range

2 `while` Loop

A **while loop** runs **as long as a condition is True**.

Loop control Statements:

Statement	Effect on Loop
break	Ends the loop completely
continue	Skips current step and continues loop
pass	Does nothing, just a placeholder

Use cases of Loops:

Loop	Purpose
`for` loop	Used when the number of iterations is known
`while` loop	Used when a condition controls the loop

Adar's Python Guide

LOOP RELATED CODE	OUTPUT

Welcome | python.py

Basic_Python > python.py > ...

```python
# for loop
for i in range(5):
    print(i)

fruits = ["apple", "banana", "mango"]
for fruit in fruits:
    print(fruit)
print("\n")

# while loop
i = 1
while(i <= 5):
    print(i)
    i += 1
print("\n")

# break statement
for i in range(10):
    if i == 5:
        break
    print(i)
print("\n")

# continue statement
for i in range(5):
    if i == 2:
        continue
    print(i)
print("\n")

# pass statement
for i in range(5):
    pass
print("\n")

# range(start, end)
for i in range(1,10):
    print(i," Adar Dey")
print("\n")

# range(start, end, increment or decrement)
for i in range(1,10,2):
    print(i," Adar Dey")
```

```
0
1
2
3
4

apple
banana
mango

1
2
3
4
5

0
1
2
3
4

0
1
3
4

1  Adar Dey
2  Adar Dey
3  Adar Dey
4  Adar Dey
5  Adar Dey
6  Adar Dey
7  Adar Dey
8  Adar Dey
9  Adar Dey

1  Adar Dey
3  Adar Dey
5  Adar Dey
7  Adar Dey
9  Adar Dey
```

```python
# For loop with else:
l = [3,23,66]
for i in l:
    print(i)
else:
    print("Done")

OUTPUT:
3
23
66
Done
```

Adar's Python

Chapter 7 – Practice Set

1. Write a program to print the **multiplication table of a given number** using a **for loop**.

2. Write a program to **greet all the person names stored in a list l** and which **starts with the letter S**.

    l = ["Harry", "Soham", "Sachin", "Rahul"]

3. Attempt **problem 1 using a while loop**.

4. Write a program to find whether a **given number is prime or not**.

5. Write a program to find the **sum of the first n natural numbers** using a **while loop**.

6. Write a program to **calculate the factorial of a given number** using a **for loop**.

7.Write a program to print the following **star pattern**:

```
*
**
***   (for n = 3)
```

8. Write a program to print the following **star pattern**:

```
  *
 ***
*****   (for n = 3)
```

9. Write a program to print the following **star pattern**:

```
* * *
*   *
* * *   (for n = 3)
```

10.Write a program to print the **multiplication table of n using for loops in reversed order**.

Ace's Python Guide

Chapter: 8
Functions and Recursions

Adar's Python Guide

Question: What is a Function in Python and write about functions in detail?

Answer:

A **function** in Python is a **block of reusable code** that performs a **specific task**.

Instead of writing the same code many times, we **put it inside a function and call it whenever needed**.

A **function is a block of code that runs when it is called to perform a specific task.**

Functions help to:

- **Reuse code**
- **Make programs organized**
- **Reduce repetition**
- **Improve readability**

```
# Basic syntax of a function:
def function_name():
    print("Inside of function")

function_name() # Function call


# Function with parameters:
def greet(name): # name is parameter
    print(f"Good morning, {name}")

greet("Adar") # Adar is argument


# Function with Return value :
def add(a,b):
    return a+b

print(add(3,4)) # output: 7


# Default argument set with parameter in a function :
def greet(start, end="Thanks"):
    print(f"{start}, {end}!")

greet("Adar", "Thank you") # output: Adar, Thank you!
greet("Ayeat") # output: Ayeat, Thanks!
```

Python Guide

Type	Description
Built-in functions	Already provided by Python (e.g., print(), len())
User-defined functions	Created by the programmer

Question: What are Recursion in Python?

Answer:

Recursion is a technique where a **function calls itself** to solve a problem.

Instead of solving the whole problem at once, the function **breaks the problem into smaller parts** and calls itself repeatedly until a **base condition** is reached.

Recursion is a programming technique in which a function calls itself until a specific condition (base case) stops the process.

```
'''
Factorial using Recursion
n!= n(n-1)(n-2)...1

'''
def fact(n):
    if(n==1 or n==0): return 1
    return n*fact(n-1)

print(fact(4)) # output: 24
print(fact(5)) # output: 120




# GCD using Recursion :
def gcd(a,b):
    if(a==0):return b
    return gcd(b%a, a)

print(gcd(4,16)) # output: 4
```

Your Task: Explore, think, draw the recursion process, and write the code for finding the factorial and GCD using recursion.

Chapter 8 – Practice Set

1. Write a program using functions to find the greatest of three numbers.

2. Write a Python program using a function to convert Celsius to Fahrenheit.

3. How do you prevent a Python print() function from printing a new line at the end?

4. Write a recursive function to calculate the sum of the first *n* natural numbers.

5. Write a Python function to print the first *n* lines of the following pattern:

```
***
**
*       (for n = 3)
```

6. Write a Python function which converts inches to centimeters.

7. Write a Python function to remove a given word from a list and strip it at the same time.

8. Write a Python function to print the multiplication table of a given number.

PROJECT 1 :

We all have played the Snake, Water, Gun game in our childhood. If you haven't, Google the rules of this game and write a Python program that can play this game with the user.

Adarsh Python Guide

Chapter: 9

File I/O

Adar's Python Guide

File I/O

The **Random Access Memory (RAM)** is **volatile**, meaning all its contents are lost once a program terminates.
In order to **store data permanently**, we use **files**.

A **file** is data stored on a **storage device** such as a hard disk. A Python program can interact with files by **reading content from them** and **writing content to them**.

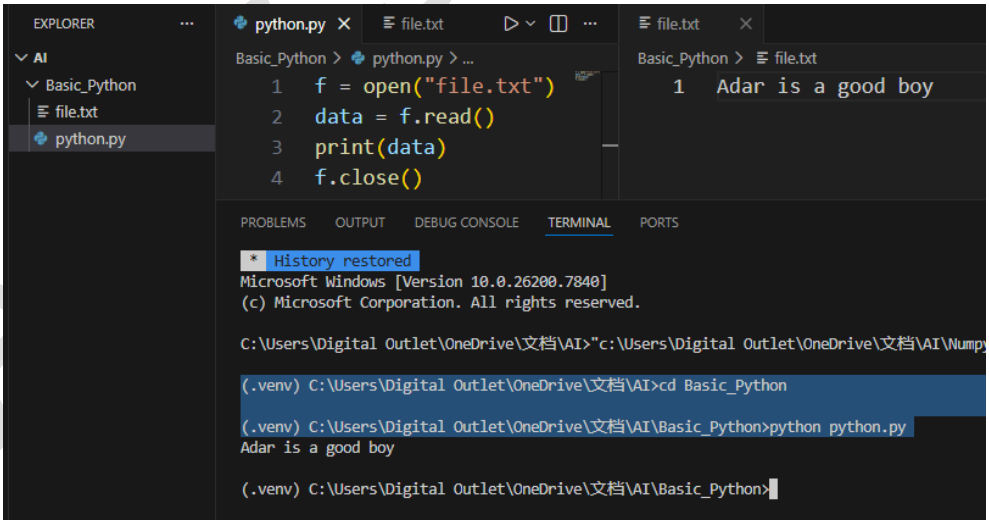
```
RAM  = Volatile (temporary storage)
HDD  = Non-volatile (permanent storage)
```

Types of Files:

There are mainly **two types of files**:

1. **Text Files**
    - Stores data in the form of **characters**.
    - Human readable.
    - Examples: `.txt`, `.py`, `.csv`
2. **Binary Files**
    - Stores data in the form of **bytes (0s and 1s)**.
    - Not human readable directly.
    - Examples: `.jpg`, `.png`, `.mp3`, `.exe`

Opening a File:

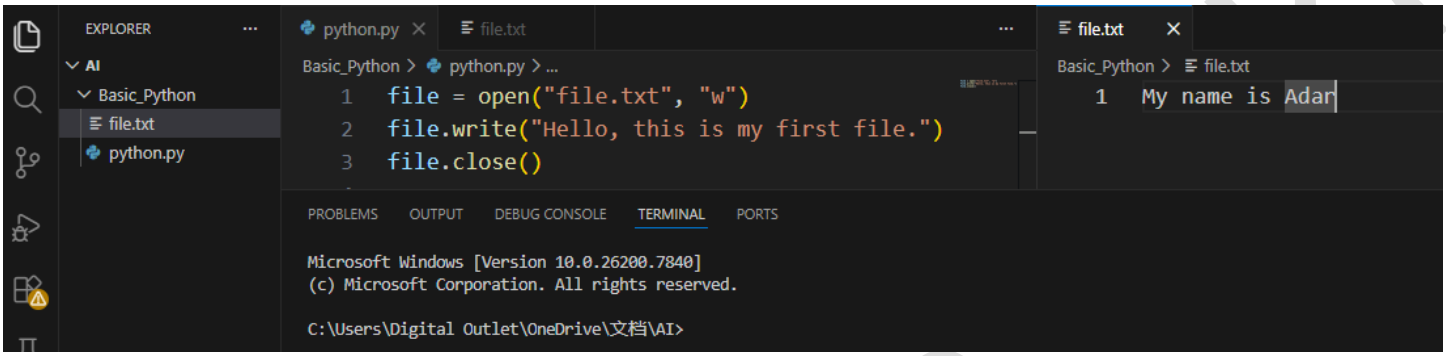


Adar's Python Guide

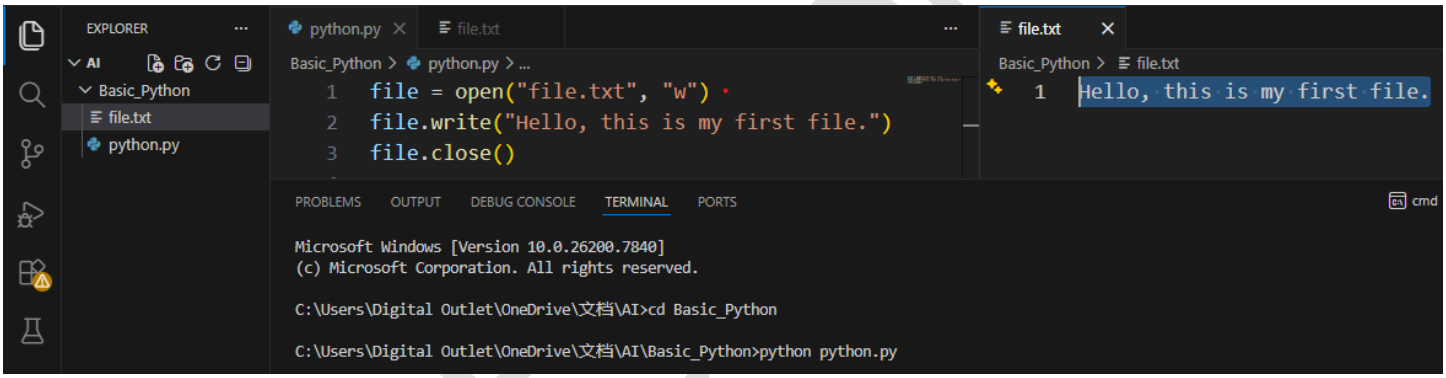
Writing to a File:

To **write data into a file**, Python uses the **write() method**.

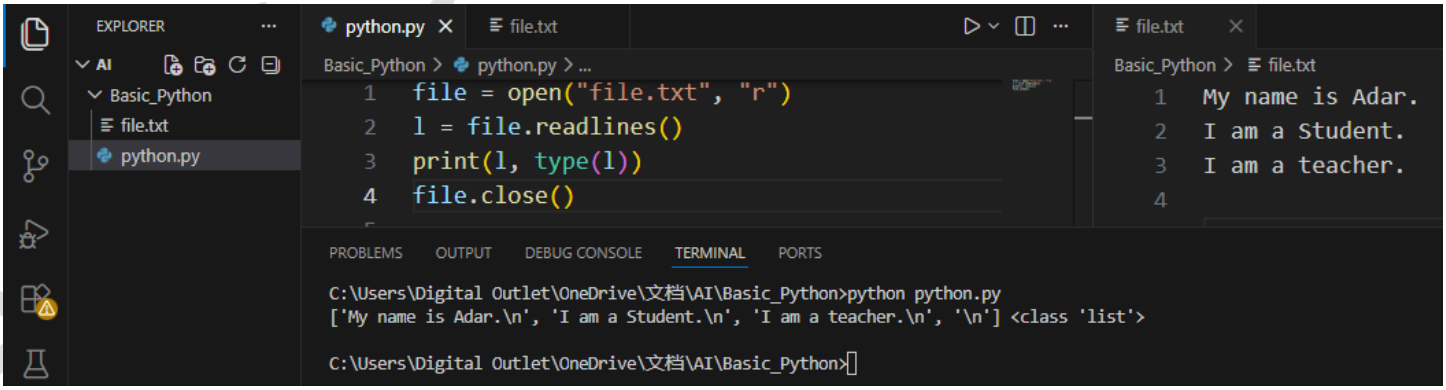
Before writing, the file must be opened in **write (w) or append (a) mode**.

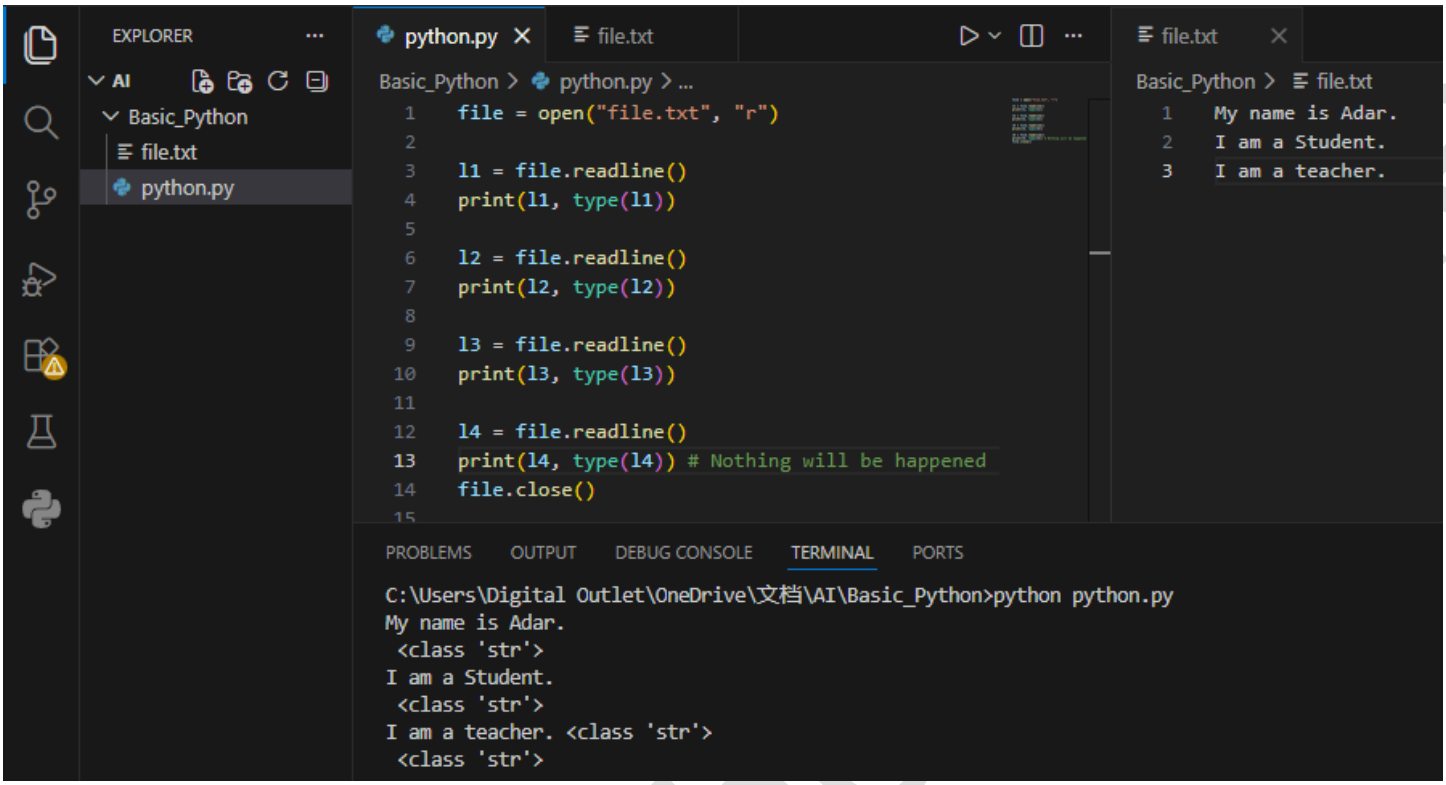


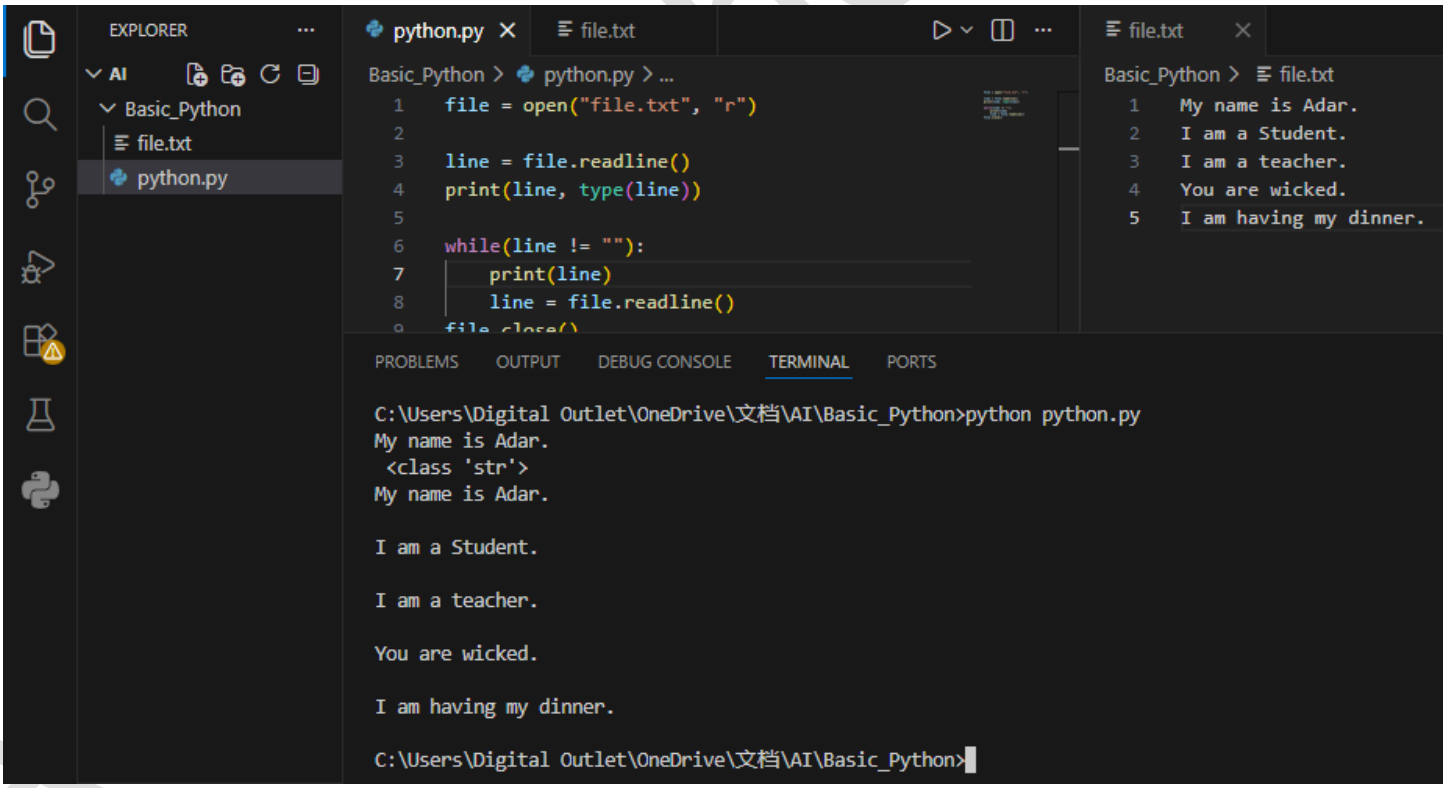
After running the `python.py` in terminal =>



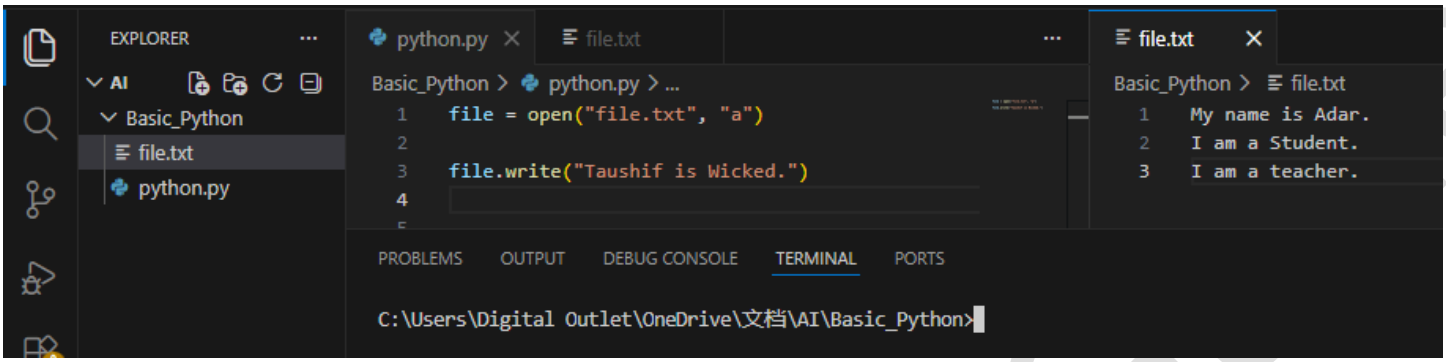
Readlines and Readline methods:

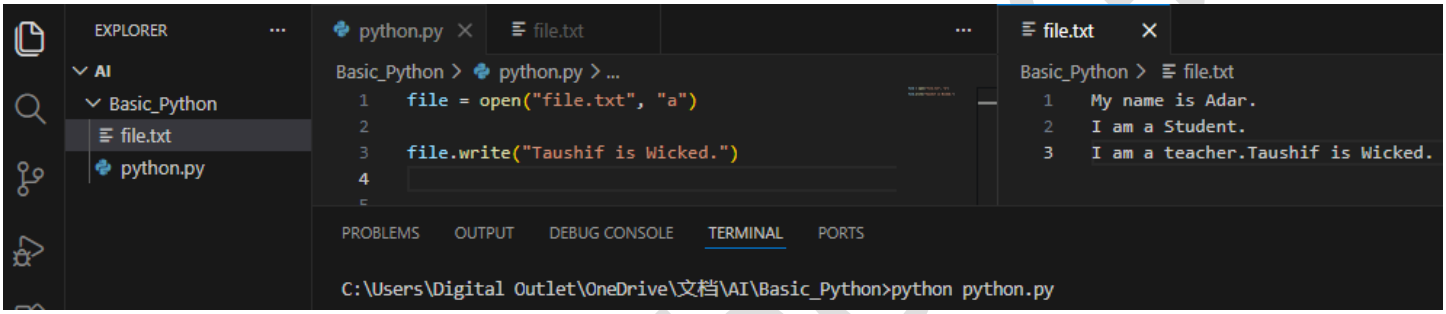



EXPLORER
AI
Basic_Python
file.txt
python.py
python.py
file.txt
Basic_Python > python.py > ...
1 file = open("file.txt", "r")
2
3 l1 = file.readline()
4 print(l1, type(l1))
5
6 l2 = file.readline()
7 print(l2, type(l2))
8
9 l3 = file.readline()
10 print(l3, type(l3))
11
12 l4 = file.readline()
13 print(l4, type(l4)) # Nothing will be happened
14 file.close()
file.txt
Basic_Python > file.txt
1 My name is Adar.
2 I am a Student.
3 I am a teacher.
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
C:\Users\Digital Outlet\OneDrive\文档\AI\Basic_Python>python python.py
My name is Adar.
<class 'str'>
I am a Student.
<class 'str'>
I am a teacher. <class 'str'>
<class 'str'>

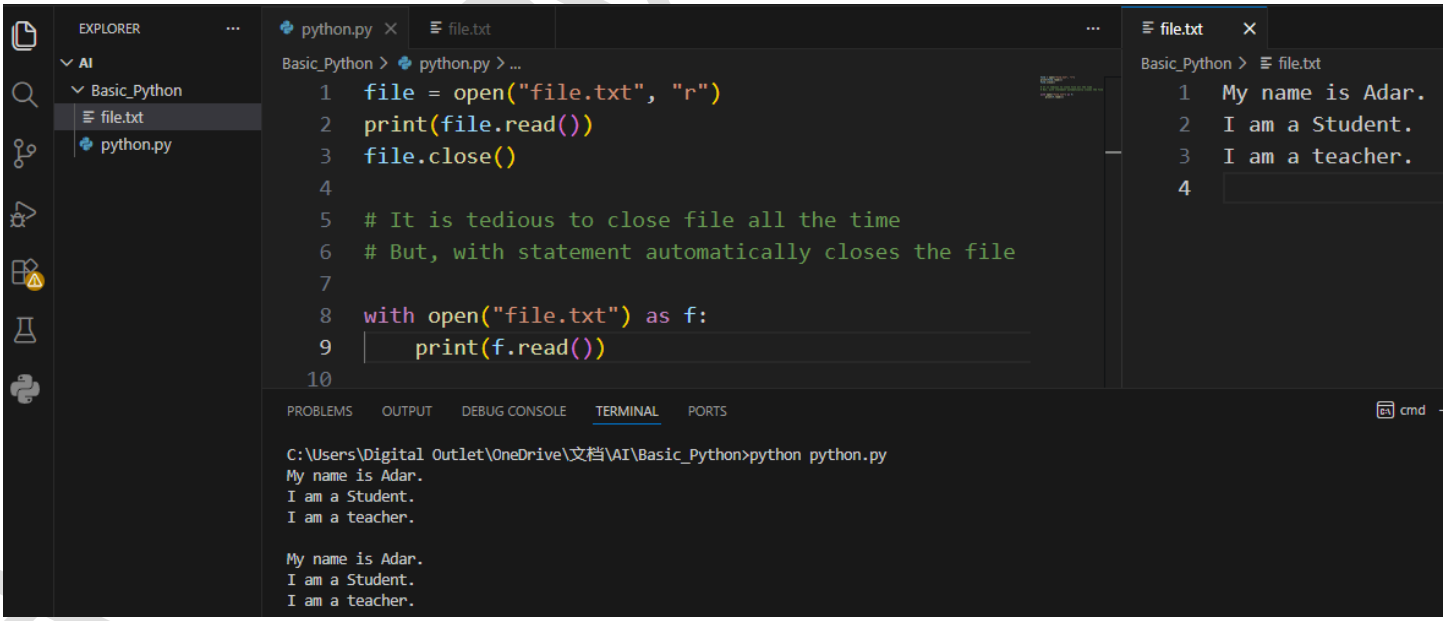

EXPLORER
AI
Basic_Python
file.txt
python.py
python.py
file.txt
Basic_Python > python.py > ...
1 file = open("file.txt", "r")
2
3 line = file.readline()
4 print(line, type(line))
5
6 while(line != ""):
7 print(line)
8 line = file.readline()
file.txt
Basic_Python > file.txt
1 My name is Adar.
2 I am a Student.
3 I am a teacher.
4 You are wicked.
5 I am having my dinner.
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
C:\Users\Digital Outlet\OneDrive\文档\AI\Basic_Python>python python.py
My name is Adar.
<class 'str'>
My name is Adar.
I am a Student.
I am a teacher.
You are wicked.
I am having my dinner.
C:\Users\Digital Outlet\OneDrive\文档\AI\Basic_Python>

Append mode in File:





With statement for opening a File:



Chapter 9 – Practice Set

1. **Write a Python program** to read the text from a file named **`poems.txt`** and determine whether it contains the word **"twinkle."**

2. The **`game()`** function in a program allows a user to play a game and returns the **score as an integer**.
Read a file named **`Hi-score.txt`**, which may either be **empty or contain the previous high score**.
Write a Python program that **updates the high score whenever the current score exceeds the previous high score**.

3. **Write a Python program** to generate **multiplication tables from 2 to 20** and store each table in **separate files**.
Place these files inside a **folder created for a 13-year-old student**.

4. A file contains the word **"Donkey"** multiple times.
Write a Python program to **replace every occurrence** of this word with **"#####"** by updating the same file.

5. **Repeat Problem 4**, but this time **use a list of words that should be censored** instead of just one word.

6. **Write a Python program** to **analyze (mine) a log file** and check whether it contains the word **"python."**

7. **Write a Python program** to **find the line number(s)** where the word **"python"** appears in the log file from **Question 6**.

8. **Write a Python program** to **create a copy of a text file named `this.txt`.**

9. **Write a Python program** to determine whether **two files are identical by comparing their contents**.

10. **Write a Python program** to **erase (wipe out) the contents of a file** using Python.

11. **Write a Python program** to **rename a file to `renamed_by_python.txt`**.

Adar's Python Guide

Chapter: 10

Programming Paradigms in Python(OOP)

Adar's Python Guide

A **programming paradigm** is a **method or style of solving problems using programming**.

Python mainly supports **four important paradigms**.

1. Procedural Programming
2. Object Oriented Programming
3. Functional Programming
4. Scripting Paradigm

Procedural Programming

Procedural programming is the **traditional programming style** where the program is written as a **sequence of instructions**.

The program executes **step by step**.

Characteristics:

- Uses **functions**
- Code runs **top to bottom**
- Focus is on **procedures (steps)**

```
def add(a, b):
    return a + b

result = add(5, 3)
print(result)
```

Here we simply **define functions and execute them**.

Languages that follow this style

- C
- Pascal
- Basic

Python also supports this style.

Adarsh Python Guide

Object Oriented Programming

Object-Oriented Programming (OOP) is a programming paradigm where programs are organized using **objects and classes** instead of only functions and procedures.

In OOP, we model real-world entities such as **students, cars, bank accounts, or users** as **objects** that contain:

- **Data (attributes / variables)**
- **Behavior (methods / functions)**

So instead of writing everything as separate functions, we **bundle data and behavior together**.

OOP concepts follow DRY(Don't Repeat Yourself) principle : **Avoid writing the same code multiple times.**

Instead, create **reusable components** such as:

- functions
- classes
- modules

OOP helps achieve **DRY** because we can reuse code through **classes, inheritance, and objects**.

```
student1_name = "Rahim"
student1_marks = 90

student2_name = "Karim"
student2_marks = 85
```

Here we repeat the same structure many times.

```
class Student:
    name = ""
    marks = 0
```

Now, Using OOP we can create many students from one **class blueprint**.

Adar Python Guide

Question: What are Class and Object in Object-Oriented Programming (OOP) in Python?

Answer:

CLASS: A **class** is a blueprint used to create objects that contain data and functions.

It defines:

- **attributes (variables)** that store data
- **methods (functions)** that define behavior

A class itself **does not represent a real entity**; it only describes how objects should be structured.

```
class Student:
    name = ""
    marks = 0
```

Here:

- Student is a **class**
- name and marks are **attributes**

OBJECT: An **object** is an **instance of a class**.

It represents a **real-world entity** created from the class blueprint.

Each object has:

- its **own data**
- access to the **methods defined in the class**

```
s1 = Student()
s1.name = "Rahim"
s1.marks = 90
```

Here:

- **s1** is an **object**
- It is created from the **Student class**

Ami's Python Guide

Key Differences:

Class	Object
Blueprint or template	Instance of a class
Defines properties	Uses those properties
Logical structure	Real entity in program

Real-Life Analogy:

1)

Concept	Example
Class	Blueprint of a house
Object	Actual house built from the blueprint

2)

Class	Object
Car	BMW, Tesla
Student	Rahim, Karim

```
class Car:              # Here, Car is Class
    brand = ""
    speed = 0

car1 = Car()            # Here, car1 is Object
car1.brand = "Toyota"
car1.speed = 120

print(car1.brand)
print(car1.speed)
```

Key Insight:

- A **class** is a blueprint that defines attributes and methods, while an **object** is an instance of that class representing a real entity in a program.

Adar Python Guide

Question: What are Attributes in OOP?

Answer:

An **attribute** is a variable inside a class that stores data about an object.

Real-Life Analogy : Think about a **Student**.

A student has properties like : `name` `roll` `age` `marks`

These properties describe the student.
In OOP, these properties are called **attributes**.

Example idea:

Class → Student

Attributes → name roll marks

There are **two main types of attributes**:

Type	Meaning
Class Attribute	Shared by all objects
Instance Attribute	Unique for each object

Example:

```python
class Student:
    college = "Comilla University"   # college → Class attribute

s1 = Student()
s1.name = "Rahim"    # name → Instance attribute

s2 = Student()
s2.name = "Karim"    # name → Instance attribute
```

```
Class → Student
     |
     |---- college (Class Attribute)
     |
Objects
   |
   |---- s1
   |      name = Rahim
   |
   |---- s2
          name = Karim
```

Python Guide

Attributes are accessed using **dot notation**. [object.attribute]

Example:

```
print(s1.name)
print(s1.college)
```

Attribute Lookup Process in Python :

When Python looks for an attribute:

Step 1 → Check object

Step 2 → Check class

Example:

```
class Student:
    college = "CU"

s1 = Student()
s1.college = "DU"  # This is also called “Attribute Overriding”
```

Now,

```
s1.college → DU
Student.college → CU    # object attribute overrides class attribute
```

Python first checks the **object**, then the **class**.

Class Attribute	Instance Attribute
Belongs to class	Belongs to object
Shared by all objects	Unique for each object
Defined inside class	Usually defined per object
Memory efficient	Stores individual data

Key Insight:

- Attributes store the data of objects in OOP, and they can be either shared by the class or unique to each instance.

Arpit's Python Guide

```python
class Student:
    college = "Comilla University"

s1 = Student()
s1.name = "Rahim"

s2 = Student()
s2.name = "Karim"

print(s1.name, s1.college) # Rahim Comilla University
print(s2.name, s2.college) # kahim Comilla University
```

Question: What is a method in OOP?

Answer:

A **method** is a function defined inside a class that describes the behavior of an object.

It defines the **behavior (actions)** of an object.

Real-Life Analogy:

Car

├── Attributes(data) → brand, speed

└── Methods(actions / behavior) → start(), stop()

Example:

```python
class Student:  # student class

    def greet(self): # greet method
        print("Hello Student")
```

```python
class Student:

    def greet(self):
        print("Hello Student")

s1 = Student()

s1.greet() # Output: Hello Student
```

Ariful's Python Guide

Why Methods Use self:

Every **instance method** must have **self as the first parameter**.

Example:

```
class Student:

    def greet(self):       # self represents the current object.
        print("Welcome")

# When you write:
s1 = Student()
s1.greet()

# Python internally does:
Student.greet(s1) # So, self becomes s1.     self = s1
```

Now combine **attributes + methods**:

```
class Student:

    def greet(self):
        print("Hello", self.name)

s1 = Student()
s1.name = "Rahim"

s2 = Student()
s2.name = "Karim"

s1.greet() # Hello Rahim
s2.greet() # Hello Karim
```

Academic

Types of Methods in Python:

Python mainly has **three types of methods**.

Type	Description
Instance Method	Works with object data
Class Method	Works with class data
Static Method	Independent utility method

Right now we focus on **instance methods**, because they are the **most fundamental**.

Key Difference Between Function and Method:

Function	Method
Independent	Inside a class
Called directly	Called through object
`func()`	`object.func()`

Question: What is a Constructor in OOP?

Answer:

A **constructor** is a special method used to initialize the attributes of an object when it is created.

Its main purpose is to **initialize the object's attributes**.

Without a constructor, we must assign attributes **manually**.

Example:

```
class Student:

    def __init__(self, name, marks):     # consturctor name must be __init__
                                         # Instance attribute (name and marks) inside constructor
        self.name = name
        self.marks = marks

    def display(self):
        print(self.name, self.marks)

s1 = Student("Rahim", 90)                # This automatically calls __init__()
s2 = Student("Karim", 85)                # This automatically calls __init__()

s1.display()       # OUTPUT: Rahim 90
s2.display()       # OUTPUT: Karim 85
```

Ashraful's Python Guide

Constructor runs **automatically** when object is created. You **do not call it manually**.

Constructor vs Normal Method:

Constructor	Normal Method
`__init__()`	Any method name
Runs automatically	Called manually
Initializes object	Performs actions

Python automatically creates a **default constructor**.

Example:

```
class Student:
    pass

# python internally provides:
Student.__init__() # But it does nothing.
```

Question: What is ENCAPSULATION in OOP?

Answer:

Encapsulation is the process of combining data and methods inside a class and controlling access to the data.

```
# Basic Example of Encapsulation:
class BankAccount:

    def __init__(self, balance):
        self.balance = balance

    def deposit(self, amount):
        self.balance = self.balance + amount

    def show_balance(self):
        print(self.balance)


# Create Object:
account = BankAccount(1000)

account.deposit(500)
account.show_balance()   # OUTPUT: 1500
```

Python Guide

balance → data

deposit() → method

show_balance() → method

Everything is **wrapped inside the class**.

The problem without Encapsulation:

```
# Without protection, someone could do:
account.balance = -100000

# This breaks the logic of the system.
#Encapsulation helps prevent this kind of misuse.
```

Python uses **special naming conventions** for encapsulation.

Type	Syntax	Meaning
Public	name	Accessible everywhere
Protected	_name	Internal use
Private	__name	Restricted access

Why Encapsulation is Important:

Encapsulation provides several benefits:

1. Data Protection: Prevents **unauthorized modification**.
2. Better Code Organization: Data and behavior stay **together**.
3. Easier Maintenance: Changes can be made **inside the class without affecting other code**.
4. Improves Security: Important in **large software systems**.

Adar's Python Guide

```python
# Example: Public, Protected, and Private Attributes in Python (Encapsulation)

class Student:

    def __init__(self, name, marks, fees, studentBankBalance):

        # Public Attributes (Accessible from anywhere)
        self.name = name
        self.marks = marks

        # Protected Attribute (Should not be accessed outside the class)
        self._dueFees = fees

        # Private Attribute (Cannot be accessed directly outside the class)
        self.__studentAccountBalance = studentBankBalance


    # Method to access private attribute
    def showBalance(self):
        print(self.__studentAccountBalance)


    # Static method (does not require self parameter)
    @staticmethod
    def greet():
        print("Hello")


# Creating an object of the Student class
s1 = Student("Rahim", 80, 12000, 350000)


# Accessing Public Attributes
print(s1.name)       # Public attribute → Accessible anywhere
print(s1.marks)      # Public attribute → Accessible anywhere


# Accessing Protected Attribute
print(s1._dueFees)   # Protected attribute → Accessible but NOT recommended outside the class


# Accessing Private Attribute (This will cause an error)
# print(s1.__studentAccountBalance)


# Private attributes cannot be accessed directly outside the class
# However, they can still be accessed using Name Mangling
print(s1._Student__studentAccountBalance)   # _ClassName__attributeName


# Accessing the private attribute through a method
s1.showBalance()


# Calling the static method
Student.greet()
```

Question: What is INHERITANCE in OOP?

Answer:

Inheritance is the mechanism by which one class can acquire the properties and methods of another class.

Real-Life Example :

Think of a **family relationship**.

Parent → Child

Animal → Dog

- Animal has **eat()**
- Animal has **sleep()**

Dog automatically gets: **eat() & sleep()**

Dog can also add: **bark()**

```
class Parent:
    pass
class Child(Parent):  # The Child class inherits from Parent class.
    pass


# Example Program:
class Animal:

    def eat(self):
        print("Animal can eat")

class Dog(Animal):

    def bark(self):
        print("Dog can bark")

d = Dog()
d.eat()   # OUTPUT: Animal can eat
d.bark()  # OUTPUT: Dog can bark
# Even though eat() is not defined in Dog, it comes from Animal.
```

Python Guide

Why Inheritence is Important:

Inheritance gives several advantages.

1. Code Reusability: You do not need to rewrite existing code.
2. Better Organization: Programs become **structured and hierarchical**.
3. Easier Maintenance: Changing parent class automatically affects child classes.

Method Overriding:

Sometimes a **child class modifies a method of the parent class**.

```
# Method Overriding:
class Animal:

    def sound(self):
        print("Animals make sound")

class Dog(Animal):

    def sound(self):
        print("Dog barks")

d = Dog()
d.sound() # OUTPUT: Dog barks
# The child method overrides the parent method.
```

Using Parent Method with `super()` :

`super()` is used in a **child class** to call methods or constructors from the **parent class**.

It is commonly used to:

- Call the **parent constructor**
- Access **parent methods**
- Avoid rewriting inherited code

Adar's Python Guide

```python
class Animal:
    def __init__(self):
        print("Animal created.")

    def sleep(self):
        print("Animal can sleep.")


class Dog(Animal):
    def __init__(self):
        super().__init__()   # Calls the parent class constructor
        print("Dog created.")

    def sleep(self):
        super().sleep()      # Calls the parent class method


d = Dog()

# OUTPUT:
# Animal created.
# Dog created.

Dog.sleep(d)

# OUTPUT:
# Animal can sleep.
```

Types of Inheritance in Python:

Python supports **five types of inheritance**.

Type	Meaning
Single Inheritance	One parent → one child
Multiple Inheritance	Multiple parents → one child
Multilevel Inheritance	Grandparent → parent → child
Hierarchical Inheritance	One parent → many children
Hybrid Inheritance	Combination of different types

Amir's Py

```python
# Single Inheritance
class A:
    pass
class B(A):
    pass


# Multilevel Inheritance
class A:
    pass
class B(A):
    pass
class C(B):
    pass


# Multiple Inheritance
class A:
    pass
class B:
    pass
class C(A, B):
    pass


# Hierarchical Inheritance
class A:
    pass
class B(A):
    pass
class C(A):
    pass
```

Your Task: Explore visual representation of several types of Inheritance.

Ada

Question: What is POLYMORPHISM in OOP?

Answer:

Polymorphism means "many forms".

In programming: **One interface → many implementations**

This means **the same function or method can behave differently depending on the object**.

Real-Life Example :

- **Human runs**
- **Car runs**
- **Software runs**

The word **run is the same**, but the **meaning changes depending on the object**. That is **polymorphism**.

```
# Example:
class Dog:
    def sound(self):
        print("Dog barks")

class Cat:
    def sound(self):
        print("Cat meows")

d = Dog()
c = Cat()

d.sound() # OUTPUT: Dog barks
c.sound() # OUTPUT: Cat meows
```

Types of Polymorphism in Python:

Type	Explanation
Method Overriding	Child class changes parent method
Operator Overloading	Same operator works differently
Duck Typing	Object type doesn't matter
Function Polymorphism	Same function works with different data

Adam Python Guide

```python
#  1️⃣ Method Overriding
# When a child class changes the method of the parent class.

class Animal:
    def sound(self):
        print("Animals make sound")

class Dog(Animal):
    def sound(self):
        print("Dog barks")

d = Dog()
d.sound()  # OUTPUT: Dog barks
# Dog overrides the Animal method


# 2️⃣ Operator Overloading
# Operators behave differently with different data types.

print(2 + 3)                # OUTPUT: 5
print("Hello " + "World")    # OUTPUT: Hello World

# Same operator (+)
# Numbers → addition
# Strings → concatenation


# 3️⃣ Function Polymorphism
# Some functions work with different types of data.

print(len("Python"))      # OUTPUT: 6
print(len([1, 2, 3, 4]))  # OUTPUT: 4
print(len((1, 2, 3)))     # OUTPUT: 3

# Same function, but works with string, list, tuple, etc.


# 4️⃣ Duck Typing (Very Pythonic Concept)
# Rule: If it walks like a duck and quacks like a duck, it is a duck.
# Python does not care about object type, only about behavior.

class Dog:
    def sound(self):
        print("Dog barks")

class Cat:
    def sound(self):
        print("Cat meows")

def make_sound(animal):
    animal.sound()

d = Dog()
c = Cat()

make_sound(d)  # OUTPUT: Dog barks
make_sound(c)  # OUTPUT: Cat meows

# Function works with any object that has sound()
# Python does not check the class type
```

Why Polymorphism is Important:

1. Flexible Code: Programs can handle **different object types easily**.
2. Code Reusability: One function works for many objects.
3. Cleaner Design: Programs become **simpler and scalable**.

Question: What is ABSTRACTION in OOP?

Answer:

Abstraction means hiding complex internal details and showing only the essential features to the user.

Hide complexity → Show only necessary functionality

Real-Life Example:

Think about a **car**.

When you drive a car, you only use:

- Steering
- Brake
- Accelerator

But you **do not see**:

- Engine combustion
- Fuel injection
- Gear mechanics
- Electrical system

All those **complex internal systems are hidden**. This is **abstraction**.

Adar's Python Guide

Python provides abstraction mainly using:

Concept	Purpose
Abstract Class	Blueprint for other classes
Abstract Method	Method without implementation

These are created using the **abc module**.

```python
from abc import ABC, abstractmethod

# Abstract Class
class Animal(ABC):

    @abstractmethod
    def sound(self):
        pass

# Child Class
class Dog(Animal):

    def sound(self):
        print("Dog Barks.")


# Creating object of child class
d = Dog()
d.sound()   # Output: Dog Barks


# Trying to create object of abstract class
# a = Animal()   # ❌ This will cause an error
# You cannot create objects of an abstract class.
```

Abstract Class

- A class that **cannot be instantiated (cannot create objects)**.
- Used as a **blueprint for other classes**.

Abstract Method

- A method declared using `@abstractmethod`.
- **Child classes must implement it**.

If a class contains at least one abstract method, the class must be declared as an abstract class.

Adar's Guide

```python
from abc import ABC, abstractmethod

# Abstract Class
class Animal(ABC):

    @abstractmethod
    def sound(self):
        print("Animal sound.")


# Child Class
class Dog(Animal):

    def sound(self):          #
        print("Dog Barks.")


# a = Animal()
# ❌ This will cause an error because we cannot create objects of an abstract class.

# Creating object of child class
d = Dog()
d.sound()   # Output: Dog Barks
```

Why Abstraction is Important:

1. Reduces Complexity: Programmers focus only on **essential features**.
2. Improves Code Structure: Large systems become **modular and manageable**.
3. Better Security: Internal logic is **hidden from the user**.
4. Improves Maintainability: You can change internal implementation **without affecting users**.

Encapsulation vs Abstraction:

Encapsulation	Abstraction
Hides data using access modifiers	Hides implementation details
Focus on data protection	Focus on design simplicity
Uses private/protected variables	Uses abstract classes

Ajar's Python

Question: What are the “Instance method”, “Class method” and “Static method” in OOP?

Answer:

```python
# Instance method (They work with individual objects)
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def show(self):  # First parameter of instance method “show()” → self
        print(self.name, self.marks)

s1 = Student("Adar", 95)
s1.show()  # OUTPUT: Adar 95


# Class method (Class methods work with the class itself, not individual objects.)
class Student:
    college = "Comilla University"

    @classmethod  # First parameter of class method “change_college()” → cls
    def change_college(cls, new_name):
        cls.college = new_name

Student.change_college("DU")
print(Student.college)  # OUTPUT: DU


# Static method (Static methods do not depend on class or object.)
# They are just utility functions placed inside the class.
class MathTools:
    @staticmethod
    def add(a, b):
        return a + b

print(MathTools.add(5, 3))  # OUTPUT: 8
```

Note:

OOP is not ending here! 🚀

If you want to crack FAANG or prepare for serious interviews, you should explore more advanced concepts like:

- Magic Methods & Dunder Methods (`__str__`, `__repr__`, `__add__`, etc.)

- Property Decorators (getter, setter, deleter)

- Composition vs Inheritance

- Metaclasses & Class Customization

*Mastering these topics will make your **OOP skills truly professional**.* 💪

Adar's Code

Chapter 10 – Practice Set

1. Create a class **Programmer** for storing information about a few programmers working at **Microsoft**.

2. Write a class **Calculator** capable of calculating the **square, cube, and square root** of a given number.

3. Create a class with a **class attribute a**. Create an object from it and set a directly using the object as **object.a = 0**.
Does this change the **class attribute**?

4. Add a **static method** to the class in **Problem 2** that prints **"Hello"**.

5. Write a class **Train** which has methods to:

- book a ticket
- get the current train status (number of seats available)
- get fare information for trains running under **Indian Railways**

6. Can you change the **self parameter inside a class method** to something else (for example "harry")? Try changing self to **slf** or **harry** and observe the effects.

7. Create a class representing a **2-D vector** and use it to create another class representing a **3-D vector**.

8. Create a class **Pets** that inherits from a class **Animals**, and then create a class **Dog** that inherits from **Pets**.
Add a method **bark()** to the **Dog** class.

9. Create a class **Employee** and add **salary** and **increment** properties to it.
Write a method **salaryAfterIncrement** using the **@property decorator**, along with a **setter** that changes the value of increment based on the salary.

10. Create a class **Complex** to represent **complex numbers**, and overload the **+** and * operators to add and multiply them.

11. Create a class **Vector** representing a vector of **n dimensions**. Overload the **+** and * operators which calculate the **sum** and **dot product** of vectors.

12. Write the **__str__()** method to print the vector in the following format: 7i + 8j + 10k
Assume the **dimension of the vector is 3** for this problem.

13. Override the **__len__()** method on the vector from **Problem 5** to display the **dimension of the vector**.

Adarsh Python Guide

PROJECT 2 :

We are going to write a program that generates a random number and asks the user to guess it.

If the player's guess is higher than the actual number, the program displays "Lower number please". Similarly, if the user's guess is too low, the program prints "Higher number please". When the user guesses the correct number, the program displays the number of guesses the player used to arrive at the number.

Hint: *Use the random module.*

Adar's Python Guide

Chapter: 11
Advanced Python

Adar's Python Guide

Walrus Operator in Python

```python
# ------------------------------------------------------------
# Walrus Operator (:=) in Python
# Introduced in Python 3.8
# Also called the "Assignment Expression Operator"
# ------------------------------------------------------------

# Definition:
# The walrus operator allows us to assign a value to a variable
# as part of an expression.

# Syntax
# variable := expression


# ------------------------------------------------------------
# Example 1: Normal Way (Without Walrus Operator)
# ------------------------------------------------------------

n = len("Python")  # First assign value
if n > 5:          # Then use the variable
    print(n)

# Here we had to calculate len() first and store it separately


# ------------------------------------------------------------
# Example 2: Using Walrus Operator
# ------------------------------------------------------------

if (n := len("Python")) > 5:
    print(n)

# Here the value of len("Python") is assigned to n
# and used immediately in the condition


# ------------------------------------------------------------
# Example 3: Using Walrus Operator in a Loop
# ------------------------------------------------------------

# Taking input until the user enters "quit"

while (user_input := input("Enter something: ")) != "quit":
    print("You entered:", user_input)

# Explanation:
# input() value is stored in user_input
# and also checked in the while condition
```

```python
# ------------------------------------------------------------
# Example 4: Using Walrus Operator with List Comprehension
# ------------------------------------------------------------

numbers = [1, 2, 3, 4, 5]

# Store the square in variable y while filtering
squares = [y for x in numbers if (y := x**2) > 10]

print(squares)

# Explanation:
# x**2 is assigned to y
# only values greater than 10 are kept



# ------------------------------------------------------------
# When to Use Walrus Operator
# ------------------------------------------------------------

# 1. To avoid repeating the same calculation
# 2. To make code shorter and more efficient
# 3. Useful in loops and conditions



# ------------------------------------------------------------
# Important Note
# ------------------------------------------------------------

# Walrus operator improves readability in some cases
# but overusing it can make code harder to read.
# Use it carefully and only when it makes the code clearer.
```

Adar's Py

Type Declaration in Python

```python
# ------------------------------------------------------------
# Type Declaration (Type Hints) in Python
# ------------------------------------------------------------
# Type hints allow us to specify the expected data type
# of variables, function parameters, and return values.
# They improve readability and help with static analysis.

# ------------------------------------------------------------
# 1. Basic Variable Type Declarations
# ------------------------------------------------------------

name: str = "Adar"      # string type
age: int = 21           # integer type
height: float = 5.8     # float type
is_student: bool = True # boolean type

print(name, age, height, is_student)


# ------------------------------------------------------------
# 2. Function Type Declarations
# ------------------------------------------------------------

def add(a: int, b: int) -> int:
    # function expects two integers and returns an integer
    return a + b

print(add(5, 7))


# ------------------------------------------------------------
# 3. List Type Declaration
# ------------------------------------------------------------

from typing import List

numbers: list[int] = [1, 2, 3, 4, 5]

print(numbers)


# ------------------------------------------------------------
# 4. Dictionary Type Declaration
# ------------------------------------------------------------

from typing import Dict

student: dict[str, int] = {
    "math": 90,
    "science": 85
}

print(student)
```

```python
# ------------------------------------------------------------
# 5. Tuple Type Declaration
# ------------------------------------------------------------

from typing import Tuple

point: tuple[int, int] = (10, 20)

print(point)


# ------------------------------------------------------------
# 6. Multiple Return Type (Tuple)
# ------------------------------------------------------------

def get_coordinates() -> tuple[int, int]:
    return (5, 10)

x, y = get_coordinates()
print(x, y)


# ------------------------------------------------------------
# 7. Optional Type (Using typing module)
# ------------------------------------------------------------

from typing import Optional

def greet(name: Optional[str]) -> None:
    if name:
        print("Hello", name)
    else:
        print("Hello Guest")

greet("Adar")
greet(None)


# ------------------------------------------------------------
# 8. Union Type (Multiple Possible Types)
# ------------------------------------------------------------

from typing import Union

def square(num: Union[int, float]) -> float:
    return num * num

print(square(5))
print(square(5.5))
```

Adar

Match-Case in Python

```python
# ------------------------------------------------------------
# Match-Case Statement in Python (Simple Example)
# Introduced in Python 3.10
# It works like a switch statement in other languages
# ------------------------------------------------------------

# Take input from user
day = int(input("Enter a number (1-3): "))

# Match-case statement
match day:

    case 1:
        print("Monday")

    case 2:
        print("Tuesday")

    case 3:
        print("Wednesday")

    case _:
        # default case (runs if no case matches)
        print("Invalid number")
```

Adar's Py

Dictionary Merge and Update Operators

```python
# ------------------------------------------------------------
# Dictionary Merge and Update Operators in Python
# ------------------------------------------------------------

# Two sample dictionaries
dict1 = {"a": 1, "b": 2}
dict2 = {"b": 3, "c": 4}

# ------------------------------------------------------------
# 1. Dictionary Merge Operator ( | )
# ------------------------------------------------------------
# Introduced in Python 3.9
# Creates a NEW dictionary by merging two dictionaries
# If keys overlap, the value from the right dictionary is used

merged_dict = dict1 | dict2

print("Merged Dictionary:", merged_dict)

# Output
# {'a': 1, 'b': 3, 'c': 4}

# ------------------------------------------------------------
# 2. Dictionary Update Operator ( |= )
# ------------------------------------------------------------
# Updates the existing dictionary in-place
# The right dictionary overwrites matching keys

dict1 |= dict2

print("Updated dict1:", dict1)

# Output
# {'a': 1, 'b': 3, 'c': 4}
```

Exception Handling in Python

```
# ------------------------------------------------------------
# Exception Handling in Python
# ------------------------------------------------------------
# Exception handling allows a program to handle runtime
# errors gracefully without crashing.
# Python provides: try, except, else, and finally blocks.
# ------------------------------------------------------------


# ------------------------------------------------------------
# 1. Basic try-except Example
# ------------------------------------------------------------

try:
    a = int(input("Enter a number: "))
    print(10 / a)

except:
    print("An error occurred")




# ------------------------------------------------------------
# 2. Handling Specific Exceptions
# ------------------------------------------------------------

try:
    a = int(input("Enter a number: "))
    result = 10 / a
    print(result)

except ZeroDivisionError as e:
    print("You cannot divide by zero.\n", f"Error Description: {e}")

except ValueError:
    print("Invalid input. Please enter a number")
```

```python
# ------------------------------------------------------------
# 3. Using else Block
# ------------------------------------------------------------
# The else block executes only when the try block
# runs successfully without any exception.

try:
    x = int(input("Enter a number: "))
    y = 10 / x

except ZeroDivisionError:
    print("Division by zero is not allowed")

else:
    print("Result is:", y)


# ------------------------------------------------------------
# 4. Using finally Block
# ------------------------------------------------------------
# The finally block always executes whether an exception
# occurs or not.

try:
    num = int(input("Enter a number: "))
    print(10 / num)

except ZeroDivisionError:
    print("Cannot divide by zero")

finally:
    print("This block always executes")


# ------------------------------------------------------------
# finally block inside a function
# ------------------------------------------------------------

def checkFinally(a, b):
    try:
        return a / b

    except Exception as e:
        print("Error is:", e)

    finally:
        print("I'm finally and I will run in any case.")
```

```python
# ------------------------------------------------------------
# 5. Raising an Exception
# ------------------------------------------------------------
# The raise keyword is used to manually trigger an exception.

age = int(input("Enter your age: "))

if age < 18:
    raise Exception("You are not eligible")

print("You are eligible")




# ------------------------------------------------------------
# Another Example of Raising an Exception
# ------------------------------------------------------------

a = int(input("Enter a: "))
b = int(input("Enter b: "))

if b == 0:
    raise ZeroDivisionError("The program does not allow division by zero")
else:
    print(a / b)




# ------------------------------------------------------------
# 6. Custom Exception Class
# ------------------------------------------------------------

class CustomError(Exception):
    pass


try:
    num = int(input("Enter a positive number: "))

    if num < 0:
        raise CustomError("Negative number not allowed")

    print("Valid number")

except CustomError as e:
    print("Error:", e)
```

Global variables in Python

```
# ------------------------------------------------------------
# Global Variables in Python
# ------------------------------------------------------------
# A global variable is declared outside any function.
# It can be accessed anywhere in the program.
# To modify a global variable inside a function,
# we must use the 'global' keyword.
# ------------------------------------------------------------



# ------------------------------------------------------------
# 1. Accessing a Global Variable inside a Function
# ------------------------------------------------------------

x = 10   # global variable

def show():
    print("Inside function:", x)   # accessing global variable

show()

print("Outside function:", x)



# Output
# Inside function: 10
# Outside function: 10




# ------------------------------------------------------------
# 2. Global Variable and Local Variable with Same Name
# ------------------------------------------------------------
# A variable defined inside a function becomes local.
# It does not change the global variable.

x = 20

def change():
    x = 50      # local variable
    print("Inside function:", x)

change()

print("Outside function:", x)



# Output
# Inside function: 50
# Outside function: 20
```

```python
# ------------------------------------------------------------
# 3. Modifying a Global Variable inside a Function
# ------------------------------------------------------------
# To permanently modify a global variable inside a function,
# we must use the 'global' keyword.

x = 100

def modify():
    global x
    x = 200
    print("Inside function:", x)

modify()

print("Outside function:", x)



# Output
# Inside function: 200
# Outside function: 200




# ------------------------------------------------------------
# 4. Creating a Global Variable inside a Function
# ------------------------------------------------------------
# A global variable can also be created inside a function
# using the 'global' keyword.

def create():
    global y
    y = 500
    print("Inside function:", y)

create()

print("Outside function:", y)



# Output
# Inside function: 500
# Outside function: 500




# ------------------------------------------------------------
# 5. Using Global Variable with Multiple Functions
# ------------------------------------------------------------
# Multiple functions can modify the same global variable.

counter = 0

def increase():
    global counter
    counter += 1
    print("Counter increased to:", counter)

def decrease():
    global counter
    counter -= 1
    print("Counter decreased to:", counter)

increase()
increase()
decrease()

print("Final counter value:", counter)



# Output
# Counter increased to: 1
# Counter increased to: 2
# Counter decreased to: 1
# Final counter value: 1
```

Enumerate Funtion in Python

```
# ------------------------------------------------------------
# enumerate() Function in Python
# ------------------------------------------------------------
# The enumerate() function is used to loop over a sequence
# (like a list, tuple, or string) while keeping track of
# the index of each element.
#
# Syntax:
# enumerate(iterable, start=0)
#
# iterable → the sequence you want to loop through
# start    → starting index (default is 0)
# ------------------------------------------------------------


# ------------------------------------------------------------
# 1. Basic Example
# ------------------------------------------------------------

fruits = ["apple", "banana", "mango"]

for item in enumerate(fruits):
    print(item)

# Output
# (0, 'apple')
# (1, 'banana')
# (2, 'mango')



# ------------------------------------------------------------
# 2. Getting Index and Value Separately
# ------------------------------------------------------------

fruits = ["apple", "banana", "mango"]

for index, value in enumerate(fruits):
    print(index, value)

# Output
# 0 apple
# 1 banana
# 2 mango
```

```python
# ------------------------------------------------------------
# 3. Changing the Starting Index
# ------------------------------------------------------------

fruits = ["apple", "banana", "mango"]

for index, value in enumerate(fruits, start=1):
    print(index, value)

# Output
# 1 apple
# 2 banana
# 3 mango



# ------------------------------------------------------------
# 4. Using enumerate() with a String
# ------------------------------------------------------------

text = "Python"

for i, ch in enumerate(text):
    print(i, ch)

# Output
# 0 P
# 1 y
# 2 t
# 3 h
# 4 o
# 5 n
```

Adar's Py

List Comprehension in Python

```
# ------------------------------------------------------------
# List Comprehension in Python
# ------------------------------------------------------------
# List comprehension is a short and powerful way to create
# a new list from an existing iterable (like list, tuple,
# range, string, etc.).
#
# Syntax:
# [expression for item in iterable if condition]
# ------------------------------------------------------------


# ------------------------------------------------------------
# 1. Basic Example
# ------------------------------------------------------------
# Create a list of numbers from 1 to 5

numbers = [i for i in range(1, 6)]
print(numbers)

# Output
# [1, 2, 3, 4, 5]



# ------------------------------------------------------------
# 2. Squaring Numbers
# ------------------------------------------------------------
# Create a list of squares

squares = [i*i for i in range(1, 6)]
print(squares)

# Output
# [1, 4, 9, 16, 25]
```

```python
# ------------------------------------------------------------
# 3. List Comprehension with Condition
# ------------------------------------------------------------
# Create a list of even numbers

evens = [i for i in range(1, 11) if i % 2 == 0]
print(evens)

# Output
# [2, 4, 6, 8, 10]



# ------------------------------------------------------------
# 4. List Comprehension with String
# ------------------------------------------------------------
# Convert each character of a string into a list

letters = [ch for ch in "Python"]
print(letters)

# Output
# ['P', 'y', 't', 'h', 'o', 'n']



# ------------------------------------------------------------
# 5. List Comprehension with Condition (Filtering)
# ------------------------------------------------------------
# Get numbers greater than 5

nums = [i for i in range(1, 11) if i > 5]
print(nums)

# Output
# [6, 7, 8, 9, 10]


# ------------------------------------------------------------
# 6. List from another List
# ------------------------------------------------------------
myList = [1,2,3,4,5,6]
squareList = [i*i for i in myList]

print(squareList)
```

Virtual Environment in Python

```
# ------------------------------------------------------------
# Virtual Environment in Python
# ------------------------------------------------------------
# A Virtual Environment is an isolated Python environment
# where packages for a specific project are installed
# separately from the global Python installation.
#
# This helps avoid version conflicts between projects.
#
# Example:
# Project A → pandas 1.5.2
# Project B → pandas latest version
#
# Both can exist on the same computer using virtual environments.
# ------------------------------------------------------------



# ------------------------------------------------------------
# Step 1 : Install virtualenv (only once)
# ------------------------------------------------------------

pip install virtualenv


# ------------------------------------------------------------
# Step 2 : Create a Virtual Environment
# ------------------------------------------------------------
# This will create a folder named "env" in the current directory.

virtualenv env


# ------------------------------------------------------------
# Step 3 : Install a Package Globally
# ------------------------------------------------------------
# Installing pandas globally (outside virtual environment)

pip install pandas==1.5.2


# ------------------------------------------------------------
# Step 4 : Check Global pandas Version
# ------------------------------------------------------------

python
import pandas
pandas.__version__

# Output
# 1.5.2
```

```
# ------------------------------------------------------------
# Step 5 : Activate the Virtual Environment (Windows PowerShell)
# ------------------------------------------------------------

.\env\Scripts\activate.ps1


# After activation, you will see something like:
# (env) PS C:\project-folder>


# ------------------------------------------------------------
# Step 6 : Install pandas inside the Virtual Environment
# ------------------------------------------------------------
# This installation will NOT affect the global Python packages.

pip install pandas


# ------------------------------------------------------------
# Step 7 : Check pandas Version inside Virtual Environment
# ------------------------------------------------------------

python
import pandas
pandas.__version__

# Output
# It will show the version installed inside "env"
# (usually the latest version)


# ------------------------------------------------------------
# Step 8 : Deactivate the Virtual Environment
# ------------------------------------------------------------

deactivate


# ------------------------------------------------------------
# Important Concept
# ------------------------------------------------------------
# Global Python Environment
# → pandas version = 1.5.2
#
# Virtual Environment (env)
# → pandas version = latest version
#
# Both environments work independently.
# Packages installed in one environment do not affect the other.
# ------------------------------------------------------------
```

pip freeze command in Python

```
# ------------------------------------------------------------
# pip freeze in Python
# ------------------------------------------------------------
# pip freeze is a command used to list all installed Python
# packages along with their exact versions.
#
# It is very useful for:
# 1. Checking installed packages
# 2. Creating requirements.txt files
# 3. Reproducing the same environment in another system
# ------------------------------------------------------------


# ------------------------------------------------------------
# 1. Check all installed packages in the current environment
# ------------------------------------------------------------

pip freeze

# Example Output
# numpy==1.26.4
# pandas==2.2.2
# matplotlib==3.9.0




# ------------------------------------------------------------
# 2. Save all installed packages into a requirements file
# ------------------------------------------------------------
# This file can later recreate the exact environment.

pip freeze > requirements.txt

# This will create a file named "requirements.txt"


# ------------------------------------------------------------
# 3. Install packages from requirements file
# ------------------------------------------------------------

pip install -r requirements.txt
```

```
# ------------------------------------------------------------
# Checking Global Python Packages
# ------------------------------------------------------------
# First make sure your virtual environment is NOT active.

deactivate

# Now run:

pip freeze

# This will show packages installed in the global Python
# environment.



# ------------------------------------------------------------
# Checking Packages in Virtual Environment
# ------------------------------------------------------------
# Activate your virtual environment first.

.\env\Scripts\activate

# Now run:

pip freeze

# This will show packages installed only inside the
# virtual environment.



# ------------------------------------------------------------
# Checking Where Packages Are Installed
# ------------------------------------------------------------
# You can also check the installation location.

pip show pandas

# Example Output
# Name: pandas
# Version: 2.2.2
# Location: C:\Users\...\env\Lib\site-packages
#
# If it shows "env\Lib\site-packages" → Virtual Environment
# If it shows "Python\Lib\site-packages" → Global Python



# ------------------------------------------------------------
# Alternative Command
# ------------------------------------------------------------
# Another command to list installed packages:

pip list
```

Lambda Function in Python

```python
# ------------------------------------------------------------
# Lambda Function in Python
# ------------------------------------------------------------
# A lambda function is a small anonymous (nameless) function
# defined using the 'lambda' keyword.
#
# It is used for short, one-line functions.
#
# Syntax:
# lambda arguments: expression
# ------------------------------------------------------------


# ------------------------------------------------------------
# 1. Basic Example
# ------------------------------------------------------------

# Normal function
def add(a, b):
    return a + b

# Lambda function
add_lambda = lambda a, b: a + b

print(add_lambda(2, 3))

# Output
# 5



# ------------------------------------------------------------
# 2. Lambda with One Argument
# ------------------------------------------------------------

square = lambda x: x * x

print(square(4))

# Output
# 16









# ------------------------------------------------------------
```

```python
# 3. Lambda inside print
# ------------------------------------------------------------

print((lambda x, y: x * y)(3, 5))

# Output
# 15



# ------------------------------------------------------------
# 4. Lambda with if-else
# ------------------------------------------------------------

check_even = lambda x: "Even" if x % 2 == 0 else "Odd"

print(check_even(6))
print(check_even(7))

# Output
# Even
# Odd



# ------------------------------------------------------------
# 5. Using lambda with map()
# ------------------------------------------------------------

nums = [1, 2, 3, 4]

squares = list(map(lambda x: x * x, nums))

print(squares)

# Output
# [1, 4, 9, 16]



# ------------------------------------------------------------
# 6. Using lambda with filter()
# ------------------------------------------------------------

nums = [1, 2, 3, 4, 5, 6]

evens = list(filter(lambda x: x % 2 == 0, nums))

print(evens)

# Output
# [2, 4, 6]
```

Join method in Python

```python
# ------------------------------------------------------------
# join() Method in Python
# ------------------------------------------------------------
# The join() method is used to combine elements of a list
# into a single string.
#
# Syntax:
# "separator".join(iterable)
# ------------------------------------------------------------


# Example 1
words = ["I", "love", "Python"]
result = " ".join(words)
print(result)

# Output
# I love Python


# Example 2
chars = ["P", "y", "t", "h", "o", "n"]
print("".join(chars))

# Output
# Python


# Example 3
nums = ["1", "2", "3"]
print("-".join(nums))

# Output
# 1-2-3
```

Adaptive

Format method in Python

```python
# ------------------------------------------------------------
# format() Method and f-String in Python
# ------------------------------------------------------------
# The format() method is used to insert values into a string.
# f-string is a modern and easier way to do the same thing.
# ------------------------------------------------------------

# ------------------------------------------------------------
# 1. Using format() Method
# ------------------------------------------------------------

name = "Adar"
age = 20

text = "My name is {} and I am {} years old.".format(name, age)
print(text)

# Output
# My name is Adar and I am 20 years old.


# ------------------------------------------------------------
# 2. Using f-String (Modern Way)
# ------------------------------------------------------------
# f-string replaces format() and is easier to read.

name = "Adar"
age = 20

text = f"My name is {name} and I am {age} years old."
print(text)

# Output
# My name is Adar and I am 20 years old.


# ------------------------------------------------------------
# 3. Multiple Values Example
# ------------------------------------------------------------

a = 5
b = 10

print("Sum of {} and {} is {}".format(a, b, a+b))
print(f"Sum of {a} and {b} is {a+b}")

# Output
# Sum of 5 and 10 is 15
# Sum of 5 and 10 is 15
```

Map, Filter & Reduce

```python
# ------------------------------------------------------------
# map(), filter(), and reduce() in Python
# ------------------------------------------------------------
# These are powerful functional programming tools used to
# process data efficiently.
# ------------------------------------------------------------

# ------------------------------------------------------------
# 1. map() Function
# ------------------------------------------------------------
# Applies a function to each element of an iterable.

nums = [1, 2, 3, 4]

# Square each number
squares = list(map(lambda x: x*x, nums))
print(squares)

# Output
# [1, 4, 9, 16]




# ------------------------------------------------------------
# 2. filter() Function
# ------------------------------------------------------------
# Filters elements based on a condition (returns True/False).

nums = [1, 2, 3, 4, 5, 6]

# Get even numbers
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens)

# Output
# [2, 4, 6]




# ------------------------------------------------------------
# 3. reduce() Function
# ------------------------------------------------------------
# Reduces the iterable into a single value.
# It is available in the functools module.

from functools import reduce

nums = [1, 2, 3, 4]

# Find product of all elements
product = reduce(lambda x, y: x * y, nums)
print(product)

# Output
# 24
```

```
# ------------------------------------------------------------
# Key Difference (Important)
# ------------------------------------------------------------
# map()    → transforms each element
# filter() → selects elements based on condition
# reduce() → combines all elements into one value
# ------------------------------------------------------------
```

Decorators in Python

```
# ------------------------------------------------------------
# DECORATORS IN PYTHON
# ------------------------------------------------------------
# A decorator is a function that modifies the behavior of
# another function without changing its actual code.
#
# It wraps a function inside another function.
# ------------------------------------------------------------



# ------------------------------------------------------------
# Basic Decorator Example
# ------------------------------------------------------------

def my_decorator(func):
    def wrapper():
        print("Before function execution")
        func()
        print("After function execution")
    return wrapper


@my_decorator
def say_hello():
    print("Hello, Adar!")

say_hello()

# Output:
# Before function execution
# Hello, Adar!
# After function execution



# ------------------------------------------------------------
# Decorator with Arguments
# ------------------------------------------------------------

def smart_divide(func):
    def wrapper(a, b):
        if b == 0:
            print("Cannot divide by zero")
            return
        return func(a, b)
    return wrapper


@smart_divide
def divide(a, b):
    print(a / b)

divide(10, 2)   # Output: 5.0
divide(10, 0)   # Output: Cannot divide by zero
```

Generators in Python

```
# ------------------------------------------------------------
# GENERATORS IN PYTHON
# ------------------------------------------------------------
# A generator is a function that returns values one by one
# using the 'yield' keyword instead of returning all at once.
#
# It saves memory and is efficient for large data.
# ------------------------------------------------------------


# ------------------------------------------------------------
# Basic Generator Example
# ------------------------------------------------------------

def count_up(n):
    i = 0
    while i < n:
        yield i
        i += 1

for num in count_up(5):
    print(num)

# Output:
# 0 1 2 3 4


# ------------------------------------------------------------
# Generator vs Normal Function
# ------------------------------------------------------------

def normal_func():
    return [1, 2, 3]

def generator_func():
    yield 1
    yield 2
    yield 3

print(normal_func())        # [1, 2, 3]
print(list(generator_func()))  # [1, 2, 3]
```

__name__ == "__main__"

```python
# ------------------------------------------------------------
# __name__ == "__main__" IN PYTHON
# ------------------------------------------------------------
# Every Python file has a special built-in variable called __name__.
#
# This variable helps us understand:
# → How the file is being used (run directly or imported)
# ------------------------------------------------------------


# ------------------------------------------------------------
# 1. What is __name__ ?
# ------------------------------------------------------------
# __name__ is a special variable automatically created by Python.
#
# It can have two values:
#
# 1. "__main__" → when the file is run directly
# 2. "filename" → when the file is imported into another file
# ------------------------------------------------------------


# ------------------------------------------------------------
# 2. Example (Run Directly)
# ------------------------------------------------------------

print(__name__)

# If you run this file directly:
# Output:
# __main__


# ------------------------------------------------------------
# 3. Example (When Imported)
# ------------------------------------------------------------
# Suppose this file name is: my_module.py
#
# In another file:
#
# import my_module
#
# Then __name__ inside my_module becomes:
# "my_module"
# ------------------------------------------------------------
```

```python
# ------------------------------------------------------------
# 4. Why do we use __name__ == "__main__" ?
# ------------------------------------------------------------
# It is used to control which code should run.
#
# Some code should only run when the file is executed directly,
# NOT when it is imported.
# ------------------------------------------------------------


# ------------------------------------------------------------
# 5. Main Example
# ------------------------------------------------------------

def greet():
    print("Hello, Adar!")


if __name__ == "__main__":
    greet()


# ------------------------------------------------------------
# Explanation:
# ------------------------------------------------------------
# Case 1: Run this file directly
# → __name__ = "__main__"
# → greet() will run
#
# Case 2: Import this file in another program
# → __name__ = "filename"
# → greet() will NOT run automatically
# ------------------------------------------------------------


# ------------------------------------------------------------
# 6. Real-World Use
# ------------------------------------------------------------
# - Writing reusable modules
# - Avoiding unwanted execution during import
# - Running test code separately
# ------------------------------------------------------------


# ------------------------------------------------------------
# 7. With main() Function (Best Practice)
# ------------------------------------------------------------

def main():
    print("This is the main function")

if __name__ == "__main__":
    main()

# This structure is widely used in real-world Python programs.
```

zip funtion in Python

```
# ------------------------------------------------------------
# ZIP FUNCTION IN PYTHON
# ------------------------------------------------------------
# zip() combines multiple iterables (like lists)
# into pairs (tuples).
# ------------------------------------------------------------

names = ["Adar", "Rahim", "Karim"]
marks = [90, 85, 88]

result = list(zip(names, marks))
print(result)

# Output:
# [('Adar', 90), ('Rahim', 85), ('Karim', 88)]

# ------------------------------------------------------------
# Loop with zip
# ------------------------------------------------------------

for name, mark in zip(names, marks):
    print(name, "scored", mark)
```

Chapter 11 – Practice Set

1. Write a program to open three files: `1.txt`, `2.txt`, and `3.txt`.
If any of these files are not present, print a message without terminating the program.

2. Write a program to print the **third, fifth, and seventh elements** from a list using the `enumerate()` function.

3. Write a **list comprehension** to generate a list containing the multiplication table of a user-entered number.

4. Write a program to display the result of `a / b`, where `a` and `b` are integers.
If `b = 0`, handle the `ZeroDivisionError` and display `"infinite"`.

5. Store the multiplication table generated in Question 3 into a file named `Tables.txt`.

6. Create two virtual environments and install a few packages in the first one.
How can you create the same environment in the second one?

7. Write a program to input a student's **name, marks, and phone number**, and format the output using the `format()` function as follows:

```
"The name of the student is Harry, his marks are 72 and phone number is 999998888"
```

8. A list contains the multiplication table of 7.
Write a program to convert it into a **vertical string format** (each number on a new line).

Adar's Python Guide

Mega Projects Ideas

1. Jarvis – Voice Assistant

Create a voice-controlled assistant that can:

- Open applications
- Search the web
- Tell time/date
- Respond to voice commands

2. Auto-Reply AI Chatbot

Build a chatbot that:

- Replies automatically to user input
- Uses basic logic or simple AI rules
- Can be extended using APIs

3. Tic-Tac-Toe Game

Develop a game that:

- Works in terminal or GUI
- Supports two players or player vs computer
- Uses conditions and game logic

4. Snake Game

Create the classic snake game using:

- Loops and conditions
- Keyboard input
- Basic graphics (like pygame)

Adar's Python Guide

5. File Organizer System

Build a tool that:

- Automatically sorts files into folders
- Uses file handling and OS module
- Organizes by file type (images, videos, docs)

6. Password Manager

Create a secure password manager that:

- Stores passwords in files
- Uses encryption (basic level)
- Retrieves saved credentials

7. Quiz Application

Develop a quiz system that:

- Asks multiple-choice questions
- Tracks score
- Reads questions from a file

8. Expense Tracker

Build an application that:

- Records daily expenses
- Saves data to a file
- Displays summary and totals

Adar's Python Guide

9. Weather App

Create a program that:

- Fetches real-time weather data
- Uses APIs
- Displays temperature and conditions

10. Mini Social Media System

Develop a simple system that:

- Allows users to create accounts
- Post messages
- Store and display data using files or dictionaries

Adar's Python Guide

Adar's Python Guide

MASTER THE ART OF CODING WITH PYTHON

WHAT YOU'LL LEARN

- Core concepts and syntax explained step-by-step
- Object-Oriented Programming made easy
- Advanced topics and modern Python techniques
- Hands-on projects to strengthen your skills
- Perfect for beginners and growing developers

Buid a strong foundation in Python and develop the skills you need to solve real-world problems with confidence.

AUTHOR

Adar Dey